

WEB3 工程师学习指南

零知识证明

SNARK、STARK、zkVM、zkML

Web3 Engineer Guide

版本 V1.0 · 2026.05

供个人学习使用 · MIT License

目录

01	目录	.
02	第 1 章 学习目标	.
03	第 2 章 直觉：零知识到底证明了什么	.
04	第 3 章 数学预备	.
05	第 4 章 多项式承诺：KZG、IPA、FRI、Brakedown、WHIR、Binius	.
06	第 5 章 证明系统全谱：从 Groth16 到 Spartan	.
07	第 6 章 算术化方言：R1CS vs PLONKish vs AIR	.
08	第 7 章 Groth16 深度剖析	.
09	第 8 章 PLONK 深度剖析	.
10	第 9 章 Halo2 深度剖析	.
11	第 10 章 Plonky2、Plonky3: Polygon 系 STARK	.
12	第 11 章 STARKs 与 S-two	.
13	第 12 章 Brakedown、Binius、WHIR：前沿三剑客	.
14	第 13 章 Bulletproofs、Marlin、Sonic、Spartan、Nova	.
15	第 14 章 算术化方言再展开：custom gates 与 lookup	.
16	第 15 章 zkVM 全景	.
17	第 16 章 zkEVM 谱系：Type 1-4	.
18	第 17 章 zkVM 实测对比(2026-04 最新数据)	.
19	第 18 章 应用版图：从 zkRollup 到 zkOracle	.
20	第 19 章 实战 1: Circom 2.2 + snarkjs 0.7 端到端	.
21	第 20 章 实战 2: Noir 1.0 等价实现 + 体验对比	.
22	第 21 章 实战 3: SP1 / Risc0 zkVM 证明斐波那契	.

23	第 22 章 练习	·
24	第 23 章 AI × ZK: zkML、opML、AI 辅助 ZK 工程的边界	·
25	第 24 章 进一步阅读	·
26	第 25 章 参考资料(含 URL)	·
27	第 26 章 隐私计算的兄弟姐妹: FHE、TEE、MPC、Garbled Circuits	·
28	第 27 章 折叠方案展开: Nova / SuperNova / HyperNova / ProtoStar / Sangria	·
29	第 28 章 Mina + o1js: 客户端零知识 webapp	·
30	第 29 章 Lurk: 内容寻址的 Lisp zkVM	·
31	第 30 章 Aztec / Penumbra / Aleo: 隐私链主网状态盘点(2026-04)	·
32	第 31 章 zk 协处理器: Brevis / Axiom / Herodotus / Lagrange / RISC Zero Steel / SP1-Reth	·
33	第 32 章 zkBridge 谱系展开	·
34	第 33 章 zkML 实测大对决(2026-04)	·
35	第 34 章 Brillig: Noir 的「不约束」逃生通道	·
36	第 35 章 ZK 工程师 2026 路线图	·
37	第 36 章 ZK 路线图与开放问题(2026-2030)	·
38	第 37 章 Powers of Tau 后续 ceremony 与 Trusted Setup 治理	·
39	第 38 章 增量参考资料(25-37 章)	·

写作日期：2026-04-27。关键依赖版本：Circom 2.2.2、snarkjs 0.7.6、SP1 Hypercube(v5)、Risc0 v3.0.5、Noir 1.0 pre-release、Halo2 PSE fork(query-collision patched)、Plonky3 2026-03 production-ready、S-two 2.0.0。

CHAPTER 01

目录

- 第 1 章 学习目标
- 第 2 章 直觉: 零知识到底证明了什么
- 第 3 章 数学预备(小白可读版)
- 第 4 章 多项式承诺: KZG、IPA、FRI、Brakedown、WHIR、Binius
- 第 5 章 证明系统全谱: Groth16 / PLONK 系 / Halo2 / STARKs / Bulletproofs / Marlin / Spartan
- 第 6 章 算术化方言: R1CS vs PLONKish vs AIR
- 第 7 章 Groth16 深度剖析
- 第 8 章 PLONK 深度剖析
- 第 9 章 Halo2 深度剖析
- 第 10 章 Plonky2、Plonky3: Polygon 系 STARK
- 第 11 章 STARKs 与 S-two
- 第 12 章 Brakedown、Binius、WHIR: 前沿三剑客
- 第 13 章 Bulletproofs、Marlin、Sonic、Spartan、Nova
- 第 14 章 算术化方言再展开: custom gates 与 lookup
- 第 15 章 zkVM 全景: Risc0 / SP1 / Jolt / Cairo VM / Valida / Lurk
- 第 16 章 zkEVM 谱系: Type 1-4 + 主流项目对照
- 第 17 章 zkVM 实测对比(2026-04 最新数据)
- 第 18 章 应用版图: 从 zkRollup 到 zkOracle
- 第 19 章 实战 1: Circom 2.2 + snarkjs 0.7 端到端
- 第 20 章 实战 2: Noir 1.0 等价实现 + 体验对比
- 第 21 章 实战 3: SP1 / Risc0 zkVM 证明斐波那契
- 第 22 章 练习
- 第 23 章 AI × ZK: zkML、opML、AI 辅助 ZK 工程的边界
- 第 24 章 延伸阅读
- 第 25 章 参考资料(含 URL)
- 第 26 章 隐私计算的兄弟姐妹: FHE、TEE、MPC、Garbled Circuits
- 第 27 章 折叠方案展开: Nova / SuperNova / HyperNova / ProtoStar / Sangria
- 第 28 章 Mina + o1js: 客户端零知识 webapp
- 第 29 章 Lurk: 内容寻址的 Lisp zkVM
- 第 30 章 Aztec / Penumbra / Aleo: 隐私链主网状态盘点(2026-04)
- 第 31 章 zk 协处理器: Brevis / Axiom / Herodotus / Lagrange / RISC Zero Steel / SP1 Reth
- 第 32 章 zkBridge 谱系展开
- 第 33 章 zkML 实测大对决(2026-04)
- 第 34 章 Brillig: Noir 的「不约束」逃生通道
- 第 35 章 ZK 工程师 2026 路线图
- 第 36 章 ZK 路线图与开放问题(2026-2030)
- 第 37 章 Powers of Tau 后续 ceremony 与 Trusted Setup 治理
- 第 38 章 增量参考资料(26-37 章)

CHAPTER 02

第1章 学习目标

2022年8月8日，OFAC把Tornado Cash列入SDN名单。一夜之间，「在区块链上保护隐私」从工程问题变成地缘问题——一段Solidity合约被宣布违法，开发者被捕，社区彻夜重估「合规友好的ZK是什么样的」。三年半后的2026年，答案叠加了一层：Aleo主网跑着Circle USDCx、Aztec Alpha上线、SP1 Hypercube用16张RTX 5090在12秒内证完一个ETH L1区块。ZK不再是「学术兴趣」，是底层基础设施，是OFAC风暴之后隐私技术的工程归宿。

前置：07-L2与扩容(zkRollup把L2状态转移正确性证给L1，依赖的就是本模块)。

具体到能力：跑完一遍Circom Poseidon原像电路、看完一份Trail of Bits Halo2 audit、读懂一段SP1 host代码——这三件事任意一件落地，本模块的目标就达到了。

读完并跑通本模块，你应当能：

1. 把Vitalik zk-SNARK三连发里的多项式承诺、双线性配对、QAP/PLONK算术化与工程实现一一对应；
2. 在Groth16 / PLONK系 / Halo2 / STARK / Bulletproofs / Marlin / Spartan / Brakedown / Binius / WHIR之间做架构选型——给出setup、proof大小、prover/verifier时间、可递归性、抗量子性、lookup支持；
3. 用Circom 2.2 + snarkjs 0.7写出Poseidon原像电路，跑完setup → witness → proof → verify，部署Solidity verifier；同一逻辑用Noir 1.0重写做对照；
4. 用SP1 / Risc0写一段Rust证明斐波那契，理解zkVM与电路DSL的差距；
5. 区分zkRollup / zkBridge / zkID / zkML / opML / private DeFi / proof-of-solvency；
6. 看ZK电路代码时立刻识别under-constrained嫌疑点(USENIX Security 2024 SoK: ≈96%真实ZK漏洞来自此类)；
7. 区分zkML / opML / AI辅助ZK工程，并对「让LLM替我写电路」保持警觉。

本模块差异化覆盖Brakedown / Binius / WHIR / Plonky3 / S-two / SP1 Hypercube / Jolt这批2024-2026新方案，每个证明系统配proof大小 / prover-verifier时间 / setup / 抗量子 / 典型用户；frozen heart与Aztec Connect多重花费等真实漏洞嵌在对应章节。生产ZK仍需读Justin Thaler Proofs, Arguments, and Zero-Knowledge并经专业审计。

CHAPTER 03

第 2 章 直觉：零知识到底证明了什么

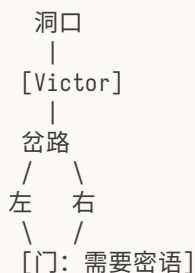
考虑一个具体问题：你想向交易所证明你拥有一个 Tornado Cash 提款资格，但又不想暴露存款时的地址。直观第一反应是「公布 hash(我的密钥)」，但这等于把密钥的指纹留给所有人去撞库。本章把这个直觉拆开，得出 ZK 的三个核心性质。

2.1 反例：哈希不是零知识

公开 $\text{hash}(x)=y$ 既不是零知识也不是知识证明：低熵 x 直接被撞库（撞库攻击就是这一现实），且公布 y 不证明你持有 x 。ZK 同时给两件事：(1) proof of knowledge——prover 真持有 witness；(2) 哪怕 witness 熵很低，proof 也不让 verifier 缩小猜测空间。

2.2 阿里巴巴山洞：交互式版本

Quisquater 1990 年的「阿里巴巴山洞」：



- Peggy 随机走左或右，Victor 随机喊「从左/右出来」；不知密语平均每两次被抓一次；
- 重复 40 次，作弊概率 $< 2^{-40} \approx 10^{-12}$ ；全程 Victor 未获任何密语信息。

抽象成现代 ZK：

- NP 语言 L (例如「存在 x 使 $\text{hash}(x)=y$ 」)；Prover 持有 witness w ，输出 proof π ；
- 三个核心性质：
- 完备性 (Completeness)：诚实 Prover 一定能说服诚实 Verifier；
- 可靠性 (Soundness)：作弊 Prover 几乎不可能让 Verifier 接受错误陈述；
- 零知识 (Zero-Knowledge)：存在一个 simulator，仅凭公共输入即可生成与真 proof 分布不可区分的假 proof——真 proof 不泄露 witness 信息。

2.3 工程上还要的两个性质

- Knowledge Soundness：存在 extractor 能从合法 prover 提取 witness (SNARK 的 K)。严格强于 soundness——后者只保证「假陈述被接受概率可忽略」，前者还要求 prover「真知道」witness；
- Succinctness：proof 短 (几十~几百 B)、verifier 快 (几 ms) (SNARK 的 S)。STARK 牺牲 succinctness (几十~几百 KB) 换无 setup + 抗量子。

2.4 非交互化：Fiat-Shamir 变换

把 Verifier 随机挑战替换为 H (所有公开消息)，前提是哈希在 ROM 里理想随机。三种典型错误模式：

- 遗漏吸收：prover 换 commitment 但 transcript 不知 → frozen heart 类(2022 TrailOfBits 在 PlonK / Bulletproofs / Spartan 实现里发现，2023 zkSync Era、gnark 等报过类似补丁)；
- 域分离失败：两个协议共用 hash → 跨协议接受假 proof；
- 可塑性：proof 被 trivial 修改后仍验过 → Tornado Cash 类 nullifier 漏洞。

工程铁律：永远不自己写 Fiat-Shamir，用已审计的 transcript 库(merlin、Halo2/Plonky3 transcript)，启动时设唯一 domain separator，每次 absorb 带字段名。

2.5 zk-SNARK 的两条直觉

1. Schwartz-Zippel + 多项式：把计算编码成多项式 $P(X)$ ，「计算正确」 $\Leftrightarrow P(X)$ 在子集上恒零 $\Leftrightarrow P(X)$ 被零多项式 $Z(X)$ 整除。Prover 承诺商 $Q(X)=P(X)/Z(X)$ ，Verifier 在随机点 r 检查 $P(r)=Z(r) \cdot Q(r)$ 。作弊要在随机 r 上对 (P,Q) 同时蒙对，概率 $\leq \deg/|F|$ 。
2. 椭圆曲线不可逆：Prover 用曲线点 $[w]$ 承诺 witness，离散对数硬度保证「满足关系者要么知道 w ，要么破了 ECDLP」。

Vitalik 三连发链路：计算 $\rightarrow$ 多项式(QAP) $\rightarrow$ 曲线点(KZG) $\rightarrow$ 配对验关系。

2.6 60 秒电梯演讲

A: 你说零知识能证明你知道我密码的哈希原像，那不就是把哈希告诉我吗？B: 不是。哈希告诉你的话，你只要拿着哈希撞库就能反推。零知识证明给你的是一个 200 字节的 proof，跟原像没有任何「可以撞」的关系。A: 那你怎么让我相信你真的知道？B: 协议会强迫我在一个你随机挑的点上对一段多项式做出回应，这段多项式只有真的知道原像才能正确算出来。我猜不到你挑哪个点，所以没法预先编造。A: proof 万一是真的假的呢？B: 作弊概率约等于 $1 / \text{一个 } 254 \text{ 比特素数}$ ，比中宇宙级彩票还小。

CHAPTER 04

第 3 章 数学预备

打开 SP1 项目第一行报错:「expected element of BabyBear field」; 第二天写 Circom 又被「prime field overflow」教做人。ZK 的数学不是装饰——它是工具链每一行报错背后的语言。本章只讲到能看懂代码的程度, 把「时钟算术」「曲线点」「多项式插值」翻译成你下次报错时能秒懂的常识。严格证明请翻 Justin Thaler Proofs, Arguments, and Zero-Knowledge。每个概念先给一个 Circom 或 SP1 里立即会撞到的工程现象, 再给数学定义。

3.1 有限域 F_p : 把数关进时钟

类比: 12 小时制时钟中 $11 + 3 = 2 \pmod{12}$ 。把 12 换成质数 p , 得到有限域 F_p 。在有限域里:

- 加法: $(a + b) \pmod{p}$, 依然封闭;
- 减法: $(a - b + p) \pmod{p}$;
- 乘法: $(a \times b) \pmod{p}$;
- 除法: $a / b = a \times b^{(-1)} \pmod{p}$, 其中 $b^{(-1)}$ 是 b 在 $\pmod{p}$ 下的乘法逆元(用扩展欧几里得算法或费马小定理 $b^{(p-2)}$ 算出来)。

质数 p 保证除 0 外每个元素都有乘法逆元(合数环如 $\pmod{12}$ 里 4 无逆元)——这是「域」区别于「环」的关键。

为什么必须是有限域? (1) 常数时间做加减乘除, 便于电路化; (2) Schwartz-Zippel 在有限域上提供核心可靠性保证。

主流 ZK 系统使用的有限域:

域	大小(bit)	来源 / 用途
BN254 标量域	254	以太坊 EVM precompile 原生支持; Circom + Groth16 + PLONK 默认使用
BLS12-381 标量域	255	Filecoin、Zcash Sapling、KZG ceremony、EIP-2537 (已上线)使用; 安全等级更高
Goldilocks 域 $p = 2^{64} - 2^{32} + 1$	64	Plonky2 主战场, 64 位友好, 乘法可用 Montgomery 优化
Mersenne-31(M31) $p = 2^{31} - 1$	31	Plonky3、StarkWare 的 S-two、Circle STARKs 主战场; 32 位 SIMD 极快
BabyBear $p = 2^{31} - 2^{27} + 1$	31	Plonky3 默认配置, SP1 使用; NTT 友好
二元塔域 $GF(2^k)$	1, 2, 4, 8, 16, 32, 64, 128	Binius / FRI-Binius; 加法即异或, 硬件极快

3.2 域选择对工程的影响

域选择直接决定 prover 的常数因子，是 2024-2026 这波性能竞赛的主战场：

- 大域(254-bit BN254)：单次乘法慢，但 EVM 配对 friendly, proof 上链便宜；
- 64-bit Goldilocks：单次乘法快，递归方便，但要 EVM 验需要 wrap 一层 Groth16；
- 31-bit M31 / BabyBear：用 32-bit SIMD (AVX-512、ARM NEON) 一次算 16 个乘法，是 SP1 Hypercube 能在 16 张 RTX 5090 上实时证 ETH L1 的根本原因；
- 二元域 $GF(2^{128})$ ：加法 = 异或，硬件电路极简；Binius 的全部赌注在「未来 ASIC 时代二元域是终极域」。

工程口诀：大域 → proof 直接上链便宜；小域 → prover 极快但需 wrap。2026 年主流：内层小域 STARK + 外层 Groth16 wrap 上链。

3.3 群与椭圆曲线

类比：逆时针旋转 30 度 = 操作 g ，做 12 次回原位 → 12 阶循环群。把点加法作为「旋转」，就得到椭圆曲线群 $E(F_p)$ 。阶是大素数 q ，离散对数问题是难的：给生成元 G 和 $P=[k] \cdot G$ ，反求 k 不可行——ECDSA、BLS、KZG 的安全基础。

椭圆曲线只需记住「点加点=点」「整数乘点=点」「给点反推标量很难」。

3.4 双线性配对

配对友好曲线提供映射 $e: G_1 \times G_2 \rightarrow G_T$ ，满足双线性性：

$$e([a] \cdot P, [b] \cdot Q) = e(P, Q)^{ab}$$

配对能验证「点之间的乘法关系」而不暴露标量。例如验证 $c=a \cdot b$ ：

$$e([a] \cdot G, [b] \cdot H) \stackrel{?}{=} e([c] \cdot G, H)$$

这是 KZG 打开证明、Groth16 verifier 的核心一步。

主流配对友好曲线：

曲线	安全等级	用途
BN254	~100 bit (受 2017 攻击降低后)	EVM precompile, alt_bn128_pairing 在 Byzantium 上线
BLS12-381	~120 bit	Zcash Sapling/Orchard、Filecoin、ETH 共识层 BLS 签名、EIP-2537
BW6-761	~120 bit	BLS12-377 的「外曲线」，在 SNARK 内部递归验证 BLS12-377 上的 SNARK
BLS12-377 + BW6-761	配对组合	Aleo、Celo、ZEXE 用过这种组合搞递归

为什么 BN254 仍是主流？EVM 唯一原生 precompile(Byzantium ecAdd / ecMul / ecPairing)。BLS12-381 的 EIP-2537 2024-2025 才上线，2026 年 Solidity 生态默认仍是 BN254。

3.5 多项式: ZK 的通用语言

两个关键性质:

- 1. 次数 d 的多项式由 $d+1$ 个点唯一确定(拉格朗日插值);
- 2. Schwartz-Zippel 引理: 两个不同 $\leq d$ 次多项式在随机点 r 重合概率 $\leq d/|F|$ 。 $|F| \approx 2^{254}$, $d \approx 2^{20}$ 时概率 $\approx 2^{-234}$ 。

ZK 的随机挑战本质都在用 Schwartz-Zippel: 承诺 $f \rightarrow$ verifier 给随机 $r \rightarrow$ 检查 $f(r)$ 。作弊换了多项式, 在 r 吻合概率可忽略。

3.6 Powers of Tau 与 KZG ceremony: toxic waste 的故事

Powers of Tau 是一种「universal trusted setup」, 对容量 $\leq N$ 的所有电路都可复用:

- 1. 选一个秘密 $\tau \in F_p$;
- 2. 算出 SRS(Structured Reference String): $(G_1, \tau \cdot G_1, \tau^2 \cdot G_1, \dots, \tau^N \cdot G_1)$ 以及 G_2 端的对应元素;
- 3. 销毁 τ (toxic waste)。

只要至少一个参与者销毁了 τ_i , 最终 $\tau = \sum \tau_i$ 就是安全的(1-of-N 信任假设)。

历史 ceremony:

ceremony	年	参与人数	用途
Sapling MPC(Zcash)	2017	~100	第一次工业级 Powers of Tau
Perpetual Powers of Tau / Hermez ptau	2019- 至 今	80+ 轮持 续	Circom 教程默认使用
Aztec Ignition	2019	~150	Aztec 早期
Filecoin Trusted Setup	2020	~200	Filecoin SnarkPack
以太坊 KZG ceremony	2023-Q1	140,000+	EIP-4844 blob commitment、Verkle Tree、 PLONK 共同 SRS

生产里直接用以太坊 KZG ceremony 的 final.ptau, 永远不要自己跑 ceremony 给主网。14 万人中只要 1 人销毁了 τ_i 即安全——1-of-N 是密码学里几乎最强的信任假设。

CHAPTER 05

第 4 章 多项式承诺: KZG、IPA、FRI、Brakedown、WHIR、Binius

把多项式承诺(PCS)想成「电话号码本指纹」: 你想让别人相信你拿着一本一万页的号码本, 但又不能把整本寄过去。你把整本的内容压成一个 32 字节的指纹发给对方, 事后对方说「翻到第 7531 页第 4 行」, 你再寄一张「这一行确实在那本里」的小卡片。验证者从头到尾没看过整本——但只要任何一页造假, 指纹就对不上。

具体例子先行: 你写了 PLONK 电路要把 witness 里的 $a/b/c$ 三列「钉死」给 verifier。每列都是几万到几百万个 field 元素, 直接发上去 prover 等于自杀。PCS 把每列压成一个 ~48 字节的曲线点(或 32 字节 Merkle root)发给 verifier, 事后 verifier 想验「第 17 行的 a 等于多少」时, prover 再发一个短打开证明。本章给五大主流 PCS 的取舍。

4.1 多项式承诺(PCS)

PCS 是现代 ZK 的核心抽象, 三个算法:

```
commit(f) → cm           // Prover 把多项式 f 压成一个短对象 cm
open(f, z) → (v, π)       // Prover 声称 f(z) = v 并附证明 π
verify(cm, z, v, π) → bool // Verifier 只看 cm, z, v, π 就能判定
```

理想性质: 承诺短(常数大小)、打开证明短、**binding**(不能改 f)、**hiding**(cm 不泄露 f)。

4.2 五大主流 PCS 对比表

方案	原理	承诺大小	打开证明大小	trusted setup	抗量子	prover 复杂度	主流使用者
KZG10	配对 + Powers of Tau	1 个 G_1 元素 ~48B	1 个 G_1 元素 ~48B	需要 (universal)	否	$O(d \log d)$	PLONK、Aztec、Scroll、EIP-4844
IPA / Bulletproofs	Pedersen + 内积论证	1 个曲线点 ~32B	$O(\log d)$ 个点	不需要	否	$O(d \log d)$	Halo2 (Zcash 默认)、Bulletproofs、Mina
FRI	Reed-Solomon 邻近性	Merkle root ~32B	$O(\log^2 d)$ hash + 域元素	不需要	是	$O(d \log d)$	STARK、Plonky2/3、SP1、Risc0
Brakedown / Ligerio	Tensor 编码 + 线性码	Merkle root	$O(\sqrt{d})$	不需要	是	$O(d)$ 线性!	Binius、部分 zkML
WHIR	受限 Reed-Solomon	Merkle root	$O(\log d)$	不需要	是	$O(d \log d)$	EUROCRYPT 2025 新方案, Whirlaway 已实现

速记: KZG 小+配对、FRI 大+哈希、Brakedown prover 线性 / proof 大、WHIR verifier 极快。Halo2 两种 PCS 都支持: IPA(无 setup, Zcash)或 KZG(需 setup, PSE/Scroll/Taiko)。

4.3 KZG10 核心: 除法构造

给定 $p(x)$, 证明 $p(z)=v$:

Step 1: 构造商多项式 $q(x) = (p(x) - v) / (x - z)$ 。注意 $p(z) - v = 0$, 所以 $(x - z)$ 整除 $(p(x) - v)$, $q(x)$ 是合法多项式(不是有理函数)。

Step 2: Prover 算出 $q(x)$, 把它承诺为 $[q] = q(\tau) \cdot G_1$, 其中 τ 是 Powers of Tau ceremony 时被销毁的「toxic waste」。

Step 3: Verifier 拿到 $[p]$ 、 $[q]$ 、 z 、 v , 做一次配对检查:

$$e([p] - [v], H) \stackrel{?}{=} e([q], [\tau] \cdot H - [z] \cdot H)$$

成立则 $p(z) = v$ 。整个打开 proof 只是一个曲线点(~48B), verifier 一次配对(~3ms)。

代价: trusted setup(ceremony 算 τ 的 Powers 并销毁)。 τ 泄露 $\rightarrow$ prover 可伪造任意打开证明。

4.4 FRI 核心: 折叠

FRI(Fast Reed-Solomon IOP of Proximity): 纯哈希承诺, 抗量子无需 setup。核心: 给定 $f: L \rightarrow F$, 说服 verifier「 f 接近低次多项式」。

```

轮 0:  $f(X) = f_{\text{even}}(X^2) + X \cdot f_{\text{odd}}(X^2)$            $\leftarrow$  拆分
轮 1: Verifier 给随机挑战  $\beta$ 
      新函数  $f'(Y) = f_{\text{even}}(Y) + \beta \cdot f_{\text{odd}}(Y)$   $\leftarrow$  次数减半
轮 2: 对  $f'$  重复 step 1
...
轮  $\log d$ : 剩下一个常数

```

每一轮 Verifier 做几个 spot check(开一些点的 Merkle 路径, 校验折叠关系)。

FRI 的可靠性近年被深入研究:

- 2022 年 Haböck 等给出了「Fiat-Shamir security of FRI」分析;
- 2024-2025 多篇 ePrint 论文细化了 round-by-round soundness;
- SP1 Hypercube 在 2026 年宣称大幅降低对 proximity gap 猜想的依赖(官方表述「conjecture-free」, 但严格意义上仍依赖 Reed-Solomon 邻近性的若干标准假设)——工程意义在于把 FRI 的安全分析从「主要靠猜想 + 实践经验」推向「以已证明的 round-by-round soundness 为主」。

FRI 代价: proof 大(每轮发 Merkle 路径, $\log^2$ hash 累计几十 KB)。换来无 setup + 抗量子。

4.5 IPA / Bulletproofs

IPA 递归折半证明 $\langle a, b \rangle = c$, 每折得一对 (L, R) Pedersen 承诺; 承诺 1 点, proof $2 \cdot \log(d)$ 点, 无需 setup, verifier $O(d)$ 。使用者: Halo2 Zcash(Pasta 曲线)、Bulletproofs(Monero/Grin)、Mina Pickles。

4.6 Brakedown: 线性时间 PCS

Golovnev-Lee-Setty-Thaler-Wahby-Wu 2021。prover $O(d)$ 线性、字段无关、proof $O(\sqrt{d})$ 、抗量子。代价: proof 大、verifier 慢。

用途: zkVM 内层 PCS、配合外层 SNARK wrap 上链、客户端证明(手机 prover)。

4.7 Binius: 二元域路线

Irreducible(前身 Ulvetanna)的 Binius 用二元塔域 $GF(2^k)$: 加法=XOR、乘法用 CLMUL 一周、塔结构可递归分解、未来 ASIC 电路比 254-bit 域小一个数量级。

2025 年发布 Binius64(客户端证明 + CPU 性能方向)。Irreducible 2024 获 Paradigm + Bain Capital Crypto 领投 \$24M A 轮。衍生工作 FRI-Binius 把 FRI 折叠嫁接到二元塔域。

Binius 是长期赌局——假设未来 ASIC 时代二元域优势显现。短期 prime field(BabyBear/M31)仍是主流。

4.8 WHIR: 超快 verifier

WHIR(Arnon-Chiesa-Fenzi-Yogev 2024, EUROCRYPT 2025): 受限 Reed-Solomon 邻近性证明。verifier 几百微秒、抗量子、无需 setup、兼任 MLE PCS。Whirlaway(LambdaClass)已接到多元 STARK。

实测: $d=2^{22}$ 时提交+打开 1.2s, 传输 63 KiB, verifier ~360 us。

4.9 Ligero / Hyrax / Dory / Basefold: 理论上知道一下

方案	主要贡献
Ligero	第一个 sublinear($\sqrt{d}$)proof 的 IOP-based PCS
Hyrax	多元线性 PCS, proof $O(\sqrt{d})$, verifier 快
Dory	透明 setup 但仍用配对, proof $O(\log d)$
Basefold	多元线性 PCS, 2024 年「FRI for multilinear」, 被 FRI-Binius 吸收
LigeSIS	2026 EUROCRYPT, 分布式友好, 性能可比 WHIR

「Brakedown 系」+「FRI 系」是 2026 年两条主路线, Binius 和 WHIR 是最新分叉。

4.10 一张图记住所有 PCS 的取舍

主轴是 proof size $\times$ verifier speed (这是上链场景最直接的两个 metric), prover 速度作为单独一列标注:



prover 速度(独立维度, 从快到慢):

- prover 极快: Brakedown / Binius(线性时间);
- prover 中等: WHIR、FRI、IPA;
- prover 较慢: KZG(需多项式 FFT + 配对预处理)。

注: WHIR verifier ~360µs, 明显快于 KZG 的 ~3ms(KZG 慢在配对运算); KZG 的优势是 proof 极小 (~48B)而非 verifier 速度。

PCS 选型对照:

约束	选
无 setup + 抗量子 + prover 极快	Brakedown / Binius
无 setup + 抗量子 + verifier 极快	WHIR
无 setup + 不抗量子 + 递归友好	IPA (Halo2 / Mina)
要 setup + 不抗量子 + proof 小	KZG (PLONK / Groth16 / Scroll)
proof 最小 (~200B)	Groth16 wrap

CHAPTER 06

第 5 章 证明系统全谱：从 Groth16 到 Spartan

2024 年 zkSync Era 的一份审计报告里写着：「PlonKup verifier gas 优化到 32 万」。同一年 Tornado Cash fork 仍在用 Groth16, verifier 25 万 gas。同一年 Starknet 在评估从 Stone 切到 S-two——proof 30 KB 但抗量子。这三家用三种不同证明系统，不是因为某家"更先进", 而是因为「电路稳不稳定 / 要不要 setup / 抗不抗量子 / proof 大小要多小」答案不同。

每个证明系统都是「算术化 + PCS + 验证协议」三件套的某种组合，差异主要在四个维度：proof 大小、verifier 时间、setup 模型、是否抗量子。本章按出现年代依次给每个系统的工程画像；§5.10 的总表是选型时直接查的那张表。

5.1 Groth16(2016)：proof 最小

Proof 3 个曲线点 (A,B,C), verifier 3 次配对。R1CS→QAP→KZG→配对验证。需要 circuit-specific phase 2 ceremony。

性能(BN254)：

指标	数值
proof 大小	~200 B(3 个 G_1 + 1 个 G_2, 业界最小)
verifier 时间	~3 ms(3 个配对)
EVM gas	~250-300k(Byzantium precompile)
trusted setup	circuit-specific phase 2, 改电路要重 ceremony
抗量子	否
递归	困难(cycle-of-curves 才行)

何时选：电路稳定 + 最小 proof + 上 EVM。Tornado Cash、Semaphore、zkSync Lite、Filecoin SnarkPack, 以及 STARK/Halo2 的外层 wrap。

5.2 PLONK(2019)：universal setup

Gabizon-Williamson-Ciobotaru 2019。Universal & updateable SRS(Powers of Tau 直接复用)+ grand product permutation argument。PLONKish 算术化：每行多列 + 自定义门 + wire-copy 通过置换乘积论证。

grand product 是 PLONK 区别于 Groth16 的核心创新，Halo2、Plonky2 均沿用。

性能(PLONK over BN254)：

指标	数值
proof 大小	~700 B - 1 KB
verifier 时间	~5 ms(少量配对)
EVM gas	~30-40 万
trusted setup	universal(直接用 ETH KZG ceremony)
抗量子	否
递归	中等(fflonk 优化后较好)

PLONK 家族变体：

- TurboPLONK：加自定义门(custom gates)；
- UltraPLONK：加查找表(lookup argument, 由 Plookup 引入)，范围检查、位运算变便宜；
- fflonk：把 verifier 配对从 ~14 个减到 2 个, gas 砍半；
- HyperPlonk：从单变量多项式改用多元线性多项式, prover 更快；
- PlonKup：调和 PLONK 和 plookup 之间的若干技术差异。

何时选：要电路升级 + lookup。Aztec Barretenberg、zkSync Era、Mina(Kimchi)、Anoma。

5.3 Halo2 (2020-)：PLONKish + IPA/KZG + 累积方案

PLONKish + IPA/KZG + accumulator scheme(递归不需昂贵配对)。

两条主线：

主线	承诺方案	是否需要 setup	主要使用者
Zcash 主线	IPA over Pasta 曲线	不需要	Zcash Orchard
PSE fork	KZG over BN254	需要(用 ETH ceremony)	Scroll、Taiko、Axiom、PSE zkEVM

累积方案：每步只把 proof「加进 accumulator」，最后一次性打开。「攒着不验，最后一起验」。

当前状态(2026-04)：2024 query collision 漏洞已 patch(版本 ≥ 2024-Q3)；PSE fork 是 EVM 友好事实标准；Trail of Bits 2025-05 发表 Axiom Halo2 deep dive 审计。

性能(PSE fork, BN254)：

指标	数值
proof 大小	~1-2 KB
verifier 时间	~10 ms
EVM gas	~30-50 万(视电路而定)
trusted setup	universal(IPA 模式无需)
抗量子	否
递归	极好(accumulator)

何时选：大型电路(zkEVM、zkML) + 递归 + custom gates。Scroll、Taiko、Axiom、EZKL 用 PSE fork。

5.4 Plonky2 / Plonky3(2022-2026)

Plonky2(2022)：Goldilocks 域 + FRI + PLONKish，无 setup、抗量子、递归快。Plonky3(2026-03 production-ready)是工具包：域/哈希/PCS/算术化均可换，默认 BabyBear + Poseidon2 + FRI。

采用者：SP1(BabyBear+Poseidon2+FRI)、Valida、Polygon Miden VM(Plonky2)。

何时选：写新 zkVM / 定制 STARK；要抗量子 + GPU 友好。

5.5 STARKs(2018)+ S-two(2026)

STARK = Scalable Transparent ARgument of Knowledge。AIR 算术化 + FRI 承诺。透明 + 后量子 + 可扩展，代价 proof 几十~几百 KB。

S-two(2026-01 上 Starknet 主网，替代 Stone)：Circle STARK + M31 域 + 递归友好。自报比 Risc0 precompile 快 28x，Stone→S-two 从分钟级降到秒级。2.0.0 fully open source on crates.io。

何时选：抗量子 + 无 setup + Cairo / Starknet。

5.6 Bulletproofs(2017)：range proof 标准

基于 IPA，无 setup，proof $O(\log n)$ (64-bit range proof ~672B)，verifier $O(n)$ 慢，不抗量子。Monero、Grin 使用。

何时选：range proof / 链下证明、可接受 verifier 慢。

5.7 Marlin / Sonic(2019)

Marlin：universal R1CS SNARK，Aleo/Anoma 早期用。Sonic：第一个 universal updateable SNARK。两者已被 PLONK 全面超越，不再推荐新项目使用。

5.8 Spartan(2020)：multilinear + sumcheck

Setty (Microsoft Research) 2020。R1CS multilinear extension + sumcheck 协议，PCS 可换(Hyrax/Brakedown/FRI)。proof ~10-50 KB，verifier sublinear。Nova 折叠方案的奠基系统。

5.9 Nova / SuperNova / HyperNova / ProtoStar: 折叠方案

Nova(2022): 把两个 R1CS 实例「折成」一个, 无需中间 SNARK, 最后一次性 SNARK。SuperNova(多电路)、HyperNova(multilinear+sumcheck)、ProtoStar(通用框架)。

何时选: 超长重复计算的增量证明、客户端隐私 dApp。

5.10 一张总表把所有证明系统串起来

系统	算术化	PCS	proof 大小	setup	抗量子	主要用户
Groth16	R1CS/QAP	KZG(电路相关)	~200 B	circuit-specific	否	Tornado/ Semaphore/ zkSync Lite
PLONK	PLONKish	KZG	~700 B-1 KB	universal	否	Aztec/Mina(变体)
UltraPLONK + Plookup	PLONKish + lookup	KZG	~1 KB	universal	否	Aztec
fflonk	PLONKish	KZG	~700 B	universal	否	优化部署版 PLONK
Halo2 (Zcash)	PLONKish	IPA	~1.5 KB	无	否	Zcash Orchard
Halo2 (PSE)	PLONKish	KZG	~1-2 KB	universal	否	Scroll/Taiko/ Axiom/EZKL
Plonky2	PLONKish + AIR	FRI	~50-200 KB	无	是	Polygon Miden
Plonky3	工具包	FRI/ Brakedown	视配置	无	是	SP1/Valida/ Polygon
STARK / S-two	AIR	FRI	~50-200 KB	无	是	Starknet
Bulletproofs	算术电路	IPA	~700 B (64-bit range)	无	否	Monero/Grin
Marlin	R1CS	KZG	~1 KB	universal	否	Aleo 早期
Spartan	R1CS multilinear	Brakedown/ Hyrax/FRI	~10-50 KB	视 PCS	视 PCS	研究
Nova/ HyperNova	R1CS folding	KZG/IPA	取决于外层	视外层	视外层	Nova Scotia/客户端证明
Binius	AIR over GF(2 ^k)	FRI-Binius	中等	无	是	Irreducible 自家产品

选系统四问：电路多大？proof 上链否？能否接受 setup？要不要抗量子？四问打勾后通常只剩一两个候选。

CHAPTER 07

第 6 章 算术化方言: R1CS vs PLONKish vs AIR

写一行 Solidity $c = a * b$ 怎么变成 ZK 约束? 这是 ZK 工程师每天要回答的最高频问题——比“哪个 PCS 更快”还高频。三种主流答案: R1CS 写成 $(1 \cdot a)(1 \cdot b) = (1 \cdot c)$ 一个三元组; PLONKish 写成一行 $q_M \cdot a \cdot b - q_0 \cdot c = 0$ 的门; AIR 写成「下一行的 $c =$ 上一行的 $a \times$ 上一行的 b 」状态转移。三种方言不是「升级版」关系——它们是不同的直觉, 分别从「线性代数视角」「自定义门 + 查表视角」「状态机视角」切入同一件事。算术化就是把高层程序翻译成这种约束的过程, 三种方言对应三类工具链。

6.1 三种方言一句话定义

- R1CS: 每条约束是 $(A \cdot z) \times (B \cdot z) = C \cdot z$, 每个约束最多一个乘法;
- PLONKish: 电路是一张多列表, 每行可定义任意自定义门(custom gate) + lookup + copy;
- AIR: 电路是一台状态机, 约束是「下一行 = f (本行)」的状态转移多项式。

6.2 R1CS: 最古老、最直白

R1CS = Rank-1 Constraint System (每个约束最多一次乘法)。形式:

对于约束矩阵 $A, B, C \in F^{m \times n}$ 和 witness 向量 $z \in F^n$ (含 1、public、private):
 $(A \cdot z) \odot (B \cdot z) = C \cdot z$ $\odot$ 是逐元素乘

例子: 要约束 $c = a \times b$, 写成 $(1 \cdot a) \times (1 \cdot b) = (1 \cdot c)$ 。

优点:

- 模型简单, 工具链最成熟(Circom、Snarky、bellman、ark-circom);
- Groth16 / Marlin / Spartan / Nova 都基于 R1CS;
- 学术资料最多。

缺点:

- 每个 bit 都要单独约束——位运算、范围检查极度昂贵;
- 没有原生 lookup, 做 SHA256 这种重位运算电路要写几万条约束。

6.3 PLONKish: 现代主流

PLONKish = PLONK 风格的算术化(社区造词)。每行有若干列:

a列	b列	c列	q_M	q_L	q_R	q_0	q_C	
1	2	3	1	0	0	-1	0	← 约束 $a * b - c = 0$
4	5	9	0	1	1	-1	0	← 约束 $a + b - c = 0$

每行是一个「门」, 由 selector polynomial(q_M 、 q_L 、 q_R 、 q_0 、 q_C 等)控制。所有门约束合在一起:

$$q_M \cdot a \cdot b + q_L \cdot a + q_R \cdot b + q_0 \cdot c + q_C = 0$$

核心扩展:

- **custom gate**: 自定义复杂门(一行做 Poseidon round);
- **lookup argument**: 有一张预定义表 T , 约束「 (a, b, c) 必须出现在 T 中某一行」——位运算、AES S-box、SHA256 都受益;
- **copy constraint via permutation**: grand product 论证保证某些 wire 相等。

主要使用者: PLONK / UltraPLONK / Halo2 / Plonky2 / Plonky3。

优点:

- 表达力强(一行做复杂操作);
- lookup 让位运算便宜得多;
- 现代 ZK 工具链(Halo2、Noir)首选。

缺点:

- 模型复杂, 新手学习曲线陡;
- 调试困难(一个约束错位整个电路 under-constrained)。

6.4 AIR: VM 风格的状态转移

AIR = Algebraic Intermediate Representation。把计算看作一台状态机:

```
state[t+1] = transition(state[t])
```

例如 Cairo VM 的状态包括「程序计数器 pc、栈指针 fp、内存」。每条指令是一个 transition rule, 写成多项式约束。

```
约束示例 (伪代码):
  下一行的 pc = 本行的 pc + opcode_size(本行的 opcode)
```

主要使用者: StarkWare(Cairo)、Polygon Miden VM、SP1/Risc0(zkVM 内部 RISC-V 是 AIR)、Plonky3。

优点:

- 对「重复执行」的程序极友好;
- prover 可以 GPU 并行化(每行独立处理);
- 适合 zkVM。

缺点:

- 通用算术约束写起来麻烦;
- 学习曲线陡(要先理解 trace polynomial、boundary constraint、transition constraint 等概念)。

6.5 三种方言的对比

维度	R1CS	PLONKish	AIR
约束形式	$a \cdot b = c$ 三元组	自定义门 + lookup + 置换	状态转移多项式
表达力	弱(每个 bit 单独约束)	强(custom gate 拼大门)	极强(适合 VM)
Lookup	不原生支持	原生支持	可加
工具链	Circom、ZoKrates、bellman	Halo2、PLONK、Plonky2/3、Noir	StarkWare AIR、Cairo、SP1 内部
证明系统	Groth16、Marlin、Spartan、Nova	PLONK 系、Halo2	STARK、Plonky2、Plonky3
证明大小	Groth16 最小	KZG: ~1KB	FRI: 几十 KB
何时选	上链成本最重要	灵活性 + lookup	写大型 VM

手写电路→Circom(R1CS)或 Noir(PLONKish); 写大型 VM→PLONKish/AIR; 最小 proof 上链→外层 Groth16 wrap。

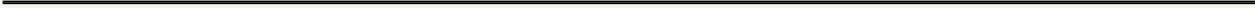
6.6 Plookup / Lasso: lookup argument 的两代

Plookup(Gabizon-Williamson 2020): 第一个 PLONK 兼容 lookup, 被 UltraPLONK、Halo2、Plonky2 吸收。

Lasso(Setty-Thaler-Wahby 2024): a16z 提出的下一代 lookup, 对结构化大表(如 RISC-V 指令集)极致优化。Jolt zkVM 的核心就是 Lasso:「整个 RISC-V 执行就是一个超大查找表」。

Twist/Shout(Jolt 2025): Lasso 的进一步优化版本, 2026 年 Jolt 主线使用。

复杂位运算 / S-box 电路必须靠 Plookup 类 lookup; 写 zkVM 直接看 Jolt 的 Lasso 是否适合你的指令集。



CHAPTER 08

第 7 章 Groth16 深度剖析

为什么 Tornado Cash 即使 2022 年被 OFAC 制裁、2024 年 fork 林立，仍然死守 Groth16？答案就 200 字节——你向 verifier 合约发的 proof 只有 3 个椭圆曲线点，gas 250-300k，这是 2016 年至今 EVM 上最便宜的 SNARK 验证。代价是：电路改一行你就要拉 5-10 个不可勾结的合作者重做一次 phase 2 ceremony——所以 Tornado Cash 八年没改过电路，这就是「最小 proof + 最痛 ceremony」的恶魔交易。

Groth16 是 proof 最小的 SNARK(~200 B)，Tornado Cash / Semaphore / zkSync Lite 直到现在仍在用它做主链验证，原因就是 EVM 上 250-300k gas 一次的硬指标。它与 PLONK 的根本差异在 circuit-specific trusted setup 与 R1CS→QAP 转换路径。

7.1 R1CS 到 QAP 的转换

QAP(Quadratic Arithmetic Program)是 Groth16 的核心算术化。给定 R1CS：

$$\forall i \in [1, m]: (A_i \cdot z)(B_i \cdot z) = (C_i \cdot z)$$

把 m 行约束分别在 m 个不同点 ($r_1, \dots, r_m$) 上插值，得到三组多项式 $\{A_i(X)\}, \{B_i(X)\}, \{C_i(X)\}$ 。整个 R1CS 满足等价于：

$$\sum z_i \cdot A_i(X) \cdot \sum z_i \cdot B_i(X) - \sum z_i \cdot C_i(X) \equiv 0 \pmod{Z(X)}$$

其中 $Z(X) = (X - r_1)(X - r_2) \cdots (X - r_m)$ 是「vanishing polynomial」。Prover 算出商多项式 $H(X) = (A \cdot B - C) / Z$ 并承诺。

7.3 双 ceremony 结构(Phase 1 + Phase 2)

- **Phase 1(universal)**：Powers of Tau，所有项目共用；
- **Phase 2(circuit-specific)**：电路 (A_i, B_i, C_i) 多项式与 SRS 结合，输出 .zkey。改一个约束就要重做 phase 2——Groth16 让位于 PLONK 的最大原因。

7.4 verifier 算法

Groth16 verifier 拿到 proof $\pi = (A, B, C)$ (A, C 是 G_1 点， B 是 G_2 点)：

$$e(A, B) \stackrel{?}{=} e([\alpha], [\beta]) \cdot e(\sum \text{public}_i \cdot [\gamma_i], [\gamma]) \cdot e(C, [\delta])$$

只要 4 次配对 + 少量 G_1 加法。EVM 上 BN254 配对 precompile 单次约 80k gas，整个验证约 25-30 万 gas。

7.5 何时选 Groth16

- 电路稳定不会改: phase 2 ceremony 一次就够;
- proof 必须最小(链上验、跨链消息、proof-of-solvency 摘要);
- 作为「内层 STARK 的外层 wrap」: SP1 / Risc0 / Halo2 → Groth16 是 2026 年的标准做法;
- 不在意 prover 慢(Groth16 prover 比 PLONK 慢约 2 倍)。

7.6 部署中的常见坑

- 公共输入数量: Groth16 verifier gas 与 public input 数量线性相关, 每多一个 public input 多 ~10k gas;
 - **trusted setup 复用陷阱**: 网上有人卖「替你做 ceremony」的服务, 绝对不要用;
 - **proof malleability**: 原始 Groth16 proof 可被简单变换(取反 B、调整 C)后仍验过——若用作 anti-replay 标识必须额外哈希;
 - **小子群攻击**: 早期 BN254 实现没做 cofactor clear, 让攻击者构造非素数阶子群上的点伪造 proof。snarkjs 0.7 已默认修复。
-

CHAPTER 09

第 8 章 PLONK 深度剖析

2019 年 Gabizon 团队发的那篇 PLONK, 名字玩了个学术冷笑话: PLONK: Permutations over Lagrange-bases for Oecumenical Noninteractive arguments of Knowledge——「一个普世的、基于拉格朗日基的置换论证」。“Oecumenical”(普世的)是这篇论文的关键词: 第一个 universal 还能 updateable 的 SNARK——你用以太坊 KZG ceremony 的 final.ptau 就能跑任何 $\leq 2^{28}$ 容量的电路, 不再需要 Groth16 那种「改电路就重 ceremony」的剧痛。

PLONK 一次解决 Groth16 的两个痛点: 电路改一行就要重 phase 2 ceremony; 不原生支持 lookup (写 Keccak 几万条 R1CS 约束)。代价是 proof 从 200B 涨到 ~1KB、验证 gas 从 25 万涨到 30-40 万 (fflonk 优化后回到 12-15 万)。

8.1 universal & updateable SRS

PLONK 的 SRS = Powers of Tau 的前若干项, 所有 $\leq 2^N$ 容量的电路共用。改电路只需重算 selector polynomial, 无需重 ceremony。直接用以太坊 KZG ceremony 的 final.ptau 即可。「Updateable」=可追加贡献者, 加第 14 万零 1 人不破坏既有安全性。

8.2 grand product permutation argument

PLONK 的算术化是 PLONKish(每行 5 列: a、b、c、selector、permutation index)。Wire-copy(这条 wire 在第 i 行的 a 列等于第 j 行的 b 列)通过一个置换 σ 表达。

要证 σ 是合法置换, PLONK 引入累积乘积 $Z(x)$:

$$\frac{Z(\omega^{i+1})}{Z(\omega^i)} = \frac{(a_i + \beta \cdot i + \gamma) \cdot (b_i + \beta \cdot (n+i) + \gamma) \cdot (c_i + \beta \cdot (2n+i) + \gamma)}{(a_i + \beta \cdot \sigma(i) + \gamma) \cdot \dots}$$

其中 ω 是 n 阶单位根, β, γ 是 verifier 随机挑战。 $Z(\omega^n) = 1$ 当且仅当 σ 是合法置换。这就是「grand product permutation argument」。

8.3 PLONK 的 verifier

verifier 收 a/b/c 三个 commitment + $Z(\text{grand product})$ + 商多项式 t 的 commitment + 打开证明。最终 2 次配对 (fflonk 之前是 12-14 次)。

8.4 PLONK 家族进化(速览)

Vanilla(2019 原版) → TurboPLONK(2020 custom gate) → UltraPLONK(2020 = Turbo + Plookup) → HyperPlonk(2022 multilinear) → fflonk(2023 verifier 2 次配对, gas ~12 万) → PlonKup / Halo2-PLONK(PLONKish 嫁接 IPA 累积)。详细变体表见第 5 章 §5.2。

PLONK 的选用：要电路升级 + lookup(哈希/范围/位运算)+ 复用 ETH KZG SRS + EVM gas 敏感 (fflonk ~12-15 万 gas)。生产用户：Aztec Barretenberg、Mina Kimchi、zkSync Era (PlonKup 变体)、Anoma、Polygon Hermez。

CHAPTER 10

第9章 Halo2 深度剖析

2020 年 Sean Bowe 给 Zcash 写 Halo2 时, 他有一个非常具体的痛点: Zcash 已经做过两轮 trusted setup(Sapling 2017、Sprout 2016), 社区每次都担心 toxic waste, 他不想做第三次。能不能在不做新 ceremony 的前提下还能跑 PLONK 风格的电路? 答案是 IPA + 累积方案——把「验 proof」这件事像滚雪球一样累积到链路终点, 最后一口气结清。这个看似学术的优化后来意外地成了 zkEVM 与 zkML 的事实底座: Scroll、Taiko、Axiom、EZKL 把 IPA 换成 KZG(依赖 ETH KZG ceremony), 跑出了 PSE fork——今天你能在以太坊上跑「Solidity 写的合约自动生成 ZK proof」这件事, 主要靠的就是这条路线。

9.1 累积方案(Accumulation Scheme)

传统递归 SNARK: proof₂ 验证 proof₁ → 每层都做完整 verifier(含配对)。Halo2 的 trick: 把验证 deferred 到最后——每层只把当前 proof「加进 accumulator」, 最后一次性把 accumulator 打开。

类比: 欠账每次记一笔, 最后一起结——每笔「欠账动作」很便宜, 递归代价远低于 cycle-of-curves 递归。

9.3 Halo2 的两条主线

- Zcash 主线(IPA over Pasta 曲线): Pasta = (Pallas, Vesta) 是一对 2-cycle 曲线, 每条的标量域恰是另一条的基域, 这让 Halo2 在自身电路里验证 Halo2 proof 极其便宜。Zcash Orchard 用这条;
- PSE fork(KZG over BN254): Privacy & Scaling Explorations 把承诺换成 KZG, proof 更小、verifier 更快、EVM 友好。Scroll、Taiko、Axiom、PSE zkEVM 都用这条。

9.4 自定义门与 lookup

Halo2 的强项是表达力:

- custom gate: 每行可以定义复杂多项式约束, 例如把 Poseidon round 写成一;
- lookup: 类 Plookup, PSE fork 增加 dynamic lookup(被查表可由 prover 提交);
- 多列 selector: 通过 Selector 类型把不同行划入不同电路区段。

9.5 Halo2 当前状态(2026-04)

- 2024 query collision soundness 漏洞: 研究者发现某些边缘电路下 Halo2 verifier 会接受非法 proof。Zcash、PSE、Axiom 三方 fork 都已 patch。新代码请确认你 fork 的版本号 $\geq$ 2024-Q3;
- Trail of Bits 的 Axiom Halo2 deep dive(2025-05): 是公开材料里对 Halo2 安全性最详细的审计文档;
- EZKL 全部基于 Halo2 PSE fork 做 zkML;
- Scroll 主网 部分电路从 Halo2 PSE 迁向新 Plonky3 backend, 但 Halo2 仍在生产。

9.6 何时选 Halo2

- 要在生产里写大型电路(zkEVM、zkML)；
- 要灵活的 custom gate + lookup；
- 要递归(accumulator 极其友好)；
- 团队有 Rust 经验。

代价：Halo2 学习曲线最陡——文档不友好，要看源码 + Trail of Bits 的审计文档才能真正掌握。

CHAPTER 11

第 10 章 Plonky2、Plonky3: Polygon 系 STARK

2022 年 Polygon 收购 Mir Protocol 时,他们瞄准的就是 Daniel Lubarov 和 Brendan Farmer 写的 Plonky2——一个把 PLONKish 跟 FRI 融在一起的怪物:既有 PLONK 的灵活性(custom gates)、又有 STARK 的抗量子,递归还能压到 200 ms 一层。四年后的 2026-03, Polygon 宣布 Plonky3 production-ready。这不是「下一代算法」——这是工具包:你想换域换哈希换 PCS? 给你三个 trait 让你随便配。SP1 用 BabyBear + Poseidon2 + FRI, Polygon 自己用另一套, Validity 又是另一套——同一个 toolkit 跑出三种 zkVM。

10.1 Plonky2 (2022)

Polygon Zero 2022 年开源:域 Goldilocks、PCS FRI、PLONKish + custom gates、内部递归 ~200 ms。Polygon Miden VM 基于 Plonky2。

10.2 Plonky3 (2026-03 production-ready)

2026-03-11 Polygon 宣布 Plonky3 production-ready。Plonky3 是工具包(toolkit),各组件均可换:

- 域: BabyBear / Goldilocks / M31; 哈希: Poseidon2 / Keccak / Blake3;
- PCS: FRI(默认) / Brakedown / TensorPCS; 算术化: PLONKish + AIR;
- 递归: STARK 折叠 STARK。

10.3 谁在用 Plonky3

- SP1(Succinct Labs): BabyBear + Poseidon2 + FRI;
- Polygon zkEVM: 从 Polygon zkEVM 退役后, Polygon 的资源转向 Plonky3 + AggLayer;
- Validity(Lita): 编译友好的 zkVM;
- 多家 rollup: 通过 Plonky3 做定制 STARK。

10.4 性能数据(2026-Q1)

- SP1 Hypercube + Plonky3: 在 16 张 NVIDIA RTX 5090 上证 ETH L1 区块, 99.7% 区块在 12 秒内完成(real-time proving);
- Plonky3 的 Polygon 配置(BabyBear + Poseidon2)已通过多家审计;
- 与 S-two(StarkWare M31 配置)相比: M31 在 SIMD 上略快, 但 BabyBear 在 NTT 上更友好。两者各有优势。

10.5 选用与不选

选: 写新 zkVM 或定制 STARK / 要抗量子 / 要 GPU 或 SIMD 加速 / Rust + 密码学背景。不选: 快速搭原型——Plonky3 是 toolkit、学习曲线陡, 先用 SP1 / Risc0。

CHAPTER 12

第 11 章 STARKs 与 S-two

具体动机: Starknet 想给一笔 Cairo 交易出证明, 但又拒绝 trusted setup (怕 toxic waste 泄露), 还要抗量子 (怕 Shor 算法十年内破 BN254)。STARK 一次满足这两个需求, 代价是 proof 从 ~200B 涨到几十 KB——所以上链时通常再外套一层 Groth16 wrap。本章给奠基论文 + 当前 Stone → S-two 的迁移。

11.1 STARK 的奠基

Eli Ben-Sasson、Iddo Bentov、Yinon Horesh、Michael Riabzev 2018 发表 Scalable, Transparent, and Post-quantum Secure Computational Integrity。STARK = Scalable Transparent ARgument of Knowledge: prover $O(n \log n)$ 、verifier $\text{polylog } n$ 、无需 trusted setup、knowledge soundness。

11.2 AIR 的工程含义

STARK 的算术化是 AIR: 把计算看作长度 N 的 trace (每行是 VM 在某一步的状态), 约束是「下一行 = transition(本行)」+ 边界约束 (开头和结尾的状态要符合声明)。

AIR 天然适合 VM: 每条 RISC-V 指令对应一组 transition rule。SP1、Risc0、Cairo VM 内部均是 AIR。

11.3 Stone Prover (2020-2026)

StarkWare 第一代生产级 prover, 2023 年开源; Starknet 主网 2020-2026 使用, prover 性能受 FFT 限制。

11.4 S-two (2026-01 上 Starknet 主网)

S-two 替代 Stone, 关键创新:

1. Circle STARK 协议: 解决传统 STARK 在小域 (M31) 上 FFT 不友好的问题;
2. M31 域: 32-bit SIMD 友好;
3. 递归友好: S-two 内部递归 proof 几秒级;
4. fully open source (crates.io 上 stwo crate)。

11.5 性能数据 (2026-Q1, StarkWare 官方)

- S-two 比 Stone 快 1 个数量级 (多场景);
- 在 Keccak chain 长度 ~3.5 的场景下, S-two 比 Risc0 precompile 快 28×;
- S-two 2.0.0 (2026-01-15 release) fully open source 上 crates.io。

11.6 选用 STARK

要抗量子 + 不要 setup + 写 Cairo / 上 Starknet + 能接受 proof 几十 KB (或外层 Groth16 wrap)。STARK 的核心 trade-off 是用 proof 体积换 transparent + 抗量子。

CHAPTER 13

第 12 章 Brakedown、Binius、WHIR：前沿三剑客

如果说 KZG 是 2017、FRI 是 2018、IPA 是 2020，那么 2021 之后冒出来的三个新名字代表了 ZK 接下来 5 年的赌局：Brakedown 押「prover 必须线性时间」、Binius 押「未来 ASIC 时代二元域 $GF(2^k)$ 通吃」、WHIR 押「verifier 几百微秒上链才便宜」。这三家分别由 a16z 投的研究团队 (Setty/Thaler/Wahby)、Paradigm + Bain 投的 Irreducible (前 Ulvetanna)、EUROCRYPT 2025 的 Arnon-Chiesa-Fenzi-Yogev 推出——背后是不同实验室对「下一代 ZK 是 prover 王、硬件王、还是 verifier 王」的三种押注。

§4.6-4.8 已给三者的核心定义与对比，本章只补 §4 之外的工程信息。

12.1 Brakedown 在工程里的位置

Tensor encoding IOP, prover $O(d)$ 线性 (KZG/FRI 都是 $O(d \log d)$)，字段无关。代价：proof $O(\sqrt{d})$ 、verifier 较慢。被用于 zkVM 内层或客户端证明，配合外层 SNARK wrap 上链。

12.2 Binius 的二元域押注

Irreducible (前身 Ulvetanna) 押二元塔域 $GF(2^k)$ ：加法即 XOR、乘法 CLMUL 一周期、塔结构可递归分解、ASIC 时代电路比 254-bit 域小一个数量级。2025 年 Binius64 把策略从「极致比特并行」转向「客户端证明 + CPU 性能」。Irreducible 2024 获 Paradigm + Bain Capital Crypto 领投 \$24M A 轮。FRI-Binius 把 FRI 折叠 + BaseFold 多元线性扩展嫁接到二元塔域，是 Binius 长期路线的递归 PCS。

12.3 WHIR 的工程数据

EUROCRYPT 2025 论文 (Arnon-Chiesa-Fenzi-Yogev)。受限 Reed-Solomon 邻近性证明 + 多元线性 PCS。与 FRI 同抗量子、同无 setup, verifier 几百微秒 (FRI 几毫秒)。实测： $d=2^{22}$ 时提交+打开 1.2s, 传输 63 KiB, verifier 360 μ s。Whirlaway (LambdaClass) 是 multilinear STARK + WHIR 的工程化实现，以太坊研究社区在评估 WHIR 的 gas 成本 (Ethereum Research 2025-Q4 主题)。

12.4 三剑客赛道定位

系统	主战场	何时关心
Brakedown	zkVM 内层、客户端证明	写自家 zkVM 内层；要字段无关
Binius	长期 ASIC 化	5-10 年远期布局；硬件团队
WHIR	gas 敏感的 STARK 方案	要 STARK 但 verifier 必须极快

CHAPTER 14

第 13 章 Bulletproofs、Marlin、Sonic、Spartan、Nova

如果你 2018-2021 年混过 Monero / Grin 社区，肯定见过 Bulletproofs 这个名字——「机密交易的 64-bit range proof 只要 672 字节」就是它带来的。如果你看 Microsoft Research 的论文，会看到 Spartan 和 Nova——前者是 Setty 2020 的「PCS 可换 + multilinear + sumcheck」，后者把它推到了「两个 R1CS 折成一个，无需中间 SNARK」的折叠时代。这一章是 ZK 历史的「中间一代」：它们不再是主流，但每一个都为后来某个范式打了地基——没有 Bulletproofs 就没有 Halo2 的 IPA 选项，没有 Spartan 就没有 Nova / HyperNova / ProtoStar 的折叠系列。

13.1 Bulletproofs (2017) : range proof 事实标准

Bünz-Bootle-Boneh-Poelstra-Maxwell-Wuille 2017。基于 IPA：

- 不需要 trusted setup；
- proof $O(\log n)$ ，64-bit range proof ~672 字节；
- verifier 时间 $O(n)$ ，慢；
- 不抗量子。

主要使用者：Monero (机密交易余额范围)、Grin、Zether、各种 ZK range proof。Solidity 上 Bulletproofs verifier 不主流 (gas 高)。

13.2 Marlin (2019)

Chiesa-Hu-Maller-Mishra-Vesely-Ward 2019, R1CS universal SNARK, 比 Sonic 快 10×；被 PLONK 全面超越后退出主流。

13.3 Sonic (2019)

Maller-Bowe-Kohlweiss-Meiklejohn 2019, 第一个工业级 universal updateable SNARK。被 Marlin 和 PLONK 超越。

13.4 Spartan (2020)

Microsoft Research 的 Srinath Setty 2020。R1CS + multilinear extension + sumcheck 协议。PCS 可换：Hyrax / Brakedown / FRI 都行。是 Nova 折叠方案的奠基系统。

13.5 Nova / SuperNova / HyperNova / ProtoStar

折叠方案系列：

- Nova (Kothapalli-Setty-Tzialla 2022)：把两个 R1CS 实例折成一个，无需中间 SNARK；
- SuperNova (2022)：支持多电路；
- HyperNova (2023)：基于 multilinear + sumcheck；
- ProtoStar (2023)：通用折叠框架；
- Nova Scotia：Circom 接进 Nova；

- Sangria(2024): PLONKish 折叠。

应用：客户端长链计算证明(每步增量)、zkVM 内层加速。Lurk 语言用 Nova 做递归。

13.6 何时选

系统	何时选
Bulletproofs	range proof; Monero 类隐私链
Marlin / Sonic	不再选 (PLONK 全面胜出)
Spartan	研究/实验; 要字段无关
Nova 系	超长重复计算的增量证明

CHAPTER 15

第 14 章 算术化方言再展开: custom gates 与 lookup

具体场景: 你想给 Tornado Cash 改一版用 Keccak 做 commitment(与 EVM 兼容), 跑 Circom 编译——R1CS 约束爆炸到 16 万条, Groth16 prover 跑了 4 分钟。同样的逻辑改写成 Halo2 + Plookup, 约束从 16 万降到几百, prover 几秒钟搞定。这中间发生了什么? 两件事: 自定义门(custom gate)让一行电路打包做 5 列计算; 查找表(lookup argument)让你免去对每个比特单独约束的代价。这一章解释为什么「lookup」是 zkVM 性能的最大杠杆。

§6 已给三种方言定义、§6.6 已介绍 Plookup / Lasso / Twist-Shout 的来源; 本章给工程含义。

14.1 为什么 R1CS 撑不住 SHA256

R1CS 每约束最多一次乘法: 完整 SHA256 约 16 万条约束(Groth16 prover 几分钟)。PLONKish + lookup 把 SHA256 降到几千、Keccak 降到几百量级。

14.2 custom gate

PLONK 默认门: $q_M \cdot a \cdot b + q_L \cdot a + q_R \cdot b + q_0 \cdot c + q_C = 0$ 。custom gate 把更多列、更复杂多项式打包成单门。Halo2 Poseidon2 round 用 5 列 $a/b/c/d/e$ + 多 selector, 一行做完 S-box + MDS 矩阵乘——原本 R1CS 几十行的逻辑被压成一行。

14.3 lookup 的工程逻辑(Plookup → Lasso → Twist/Shout)

- Plookup(2020): 用 sorted-by-T 辅助列 s , 论证 witness 列 f 与表 T 的多重集关系; 被 UltraPLONK / Halo2 / Plonky2 吸收;
- Lasso(2024): a16z 的 insight——结构化大表(如 2^{32} RISC-V 指令集)可用 sumcheck + sparse poly commit, 把代价从 $O(|T|)$ 降到 $O(|f|)$; Jolt zkVM 的赌注;
- Twist/Shout(Jolt 2025): Twist 把一类 lookup 转成更紧的 product 论证; Shout 批量化 lookup 配 GPU; Jolt 2026 主线已切此配置。

关心 lookup 的场景: 写 zkVM / SHA256-Keccak-AES 类位运算 / zkEVM 字节码 / 要 GPU 加速。
lookup 是 zkVM 的关键性能杠杆。

CHAPTER 16

第 15 章 zkVM 全景

2024 年 Jolt 团队在 a16z crypto 博客发了张图, 把所有 zkVM 排在一起: SP1、Risc0、Jolt 都是 RISC-V, Cairo 是自定义 ISA, Lurk 是 Lisp。意思是说:「写普通 Rust, 让编译器把它变成 RISC-V, 然后 zkVM 自动把每条指令的执行做成 AIR 约束, 最后吐出一个 proof。」这是 ZK 工程从「电路工匠」到「应用程序员」的临界拐点——SP1 Hypercube 在 16 张 RTX 5090 上证一个 ETH L1 区块只要 12 秒, 意味着「real-time proving」从论文术语变成生产里程碑。

zkVM: 写普通 Rust(或 C/Go) → 编译成 RISC-V 字节码 → VM 把每条指令的执行轨迹做成 AIR 约束自动出 proof。代价是 prover 比手写电路慢一个数量级(2024-2026 已压到 3-5 倍)。抽象层次: 手写电路 $\approx$ 汇编; zkVM $\approx$ C; zkEVM $\approx$ 业务方根本不知道有 ZK。

15.2 RISC Zero(2022-)

第一个开源 RISC-V zkVM。

- 指令集: RISC-V RV32IM;
- 域: BabyBear;
- 证明系统: STARK(Plonky3 类) + Groth16 wrap;
- 最新版: risc0-zkvm 3.0.5(2026-Q1);
- 路线图: ROVM 2.0, 目标形式化验证(Veridise + Picus) + 加 Keccak / ECDSA / pairing / BLS12-381 precompile + RSA accelerator。

性能(zkbenchmarks.com, 2026-Q1):

- Fibonacci 1000 万 cycles: 19m10s(CPU);
- 加 GPU 后: 约 2-3 倍提速;
- proof 大小: ~300 KB(裸 STARK) / ~250 B(Groth16 wrap 后)。

15.3 SP1(Succinct Labs, 2024-)

SP1 是 2026 年 zkVM 性能竞赛的领跑者:

- 指令集: RISC-V RV32IM;
- 域: BabyBear;
- 证明系统: Plonky3 STARK + 可选 Groth16 wrap;
- 形式化验证: 与 Nethermind + Ethereum Foundation 合作完成 RISC-V 约束的完整形式化验证;
- 审计: Veridise / Cantina / Zellic / KALOS;
- 预编译: keccak256、sha256、secp256k1、ed25519、bn254、bls12-381(2026 加 pairing)。

SP1 Hypercube(2026-Q1 release):

- 16 张 NVIDIA RTX 5090 上证 ETH L1 区块, 99.7% 区块在 12 秒内完成;
- 在 FRI 类 zkVM 里第一次把 proximity gap 猜想的依赖降到最小(官方称 "conjecture-free FRI", 仍依赖 Reed-Solomon 邻近性的若干标准假设);

- 2025-05 实测：以太坊 block 22309250(143 笔交易, 3200 万 gas)证明用时 10.8 秒, proof 1MB。

15.4 Jolt(a16z, 2024-)

a16z crypto 出品, 完全靠 lookup 而不是算术约束:

- 指令集: RISC-V RV32I;
- PCS: Dory(KZG 类, 多元线性);
- 算术化: sumcheck + Lasso lookup + Twist/Shout;
- 状态: alpha, 不要在生产用;
- 性能: 2026-02 preliminary benchmarks 自报比 Risc0 快 5×、比 SP1 快 2×; 正式版本对决待 Hypercube 之后再。

形式化验证亮点: Lasso 已被 ACL2 定理证明系统形式化验证(RV32I 全部 32-bit 指令)。

15.5 Cairo VM / S-two(StarkWare)

Cairo 是 StarkWare 的 zk 友好语言 + VM:

- 指令集: Cairo asm(自定义, 不是 RISC-V);
- 域: M31;
- 证明系统: S-two 2.0.0(Circle STARK);
- Starknet 主网默认;
- 与 RISC-V 系 zkVM 不同: Cairo 不假装是 EVM 兼容, 而是设计成「ZK 原生」。

15.6 Valida(Lita)

基于 Plonky3 的实验性 zkVM, 自定义指令集, 编译/GPU 友好; 2026 年仍实验阶段, 不在生产用。

15.7 Lurk

静态作用域 Lisp dialect; 证明系统早期 Nova + Halo2, 2025 年起切 Sphinx(SP1 fork); 维护方 Argument(前身 Lurk Lab); 用例: Filecoin FVM、IPFS 完整性证明、递归 SNARK 实验。

15.8 OpenVM、Pico、ZKM、Nexus、Novanet

2025-2026 涌现的新 zkVM: OpenVM(模块化框架)、Pico(实验性, Brevis 系)、ZKM(MIPS)、Nexus / Novanet(folding 路线)。Fenbushi 2025-06 横评(SP1、Risc0、OpenVM、Pico、ZKM、Jolt、Nexus、Novanet): SP1+GPU 扩展性最佳; Risc0 / OpenVM 中等; Pico / Jolt / ZKM 在大计算下时间显著上升。

15.9 zkVM vs 手写电路 vs zkEVM

- 手写电路(Circom / Noir / Halo2): 自由度最高、prover 最快、bug 风险最高;
- zkVM(SP1 / Risc0 / Jolt): 写 Rust, 每条 RISC-V 指令付 ZK 税;
- zkEVM(Linea / Scroll / Polygon / Taiko): 业务方写 Solidity 直接跑, 整个 EVM 状态机被做成电路。

CHAPTER 17

第 16 章 zkEVM 谱系: Type 1-4

2022 年 8 月 Vitalik 发布 The different types of ZK-EVMs 那天, 业界混乱程度达到顶峰: Polygon、zkSync、Scroll、Starknet 各自宣称「我们是最 EVM 兼容的 zkEVM」, 但定义全不一样。Vitalik 一篇博文把这片混沌切成 4 类——从 Type 1(完全等价以太坊, 连 Merkle Patricia Trie 都不改)到 Type 4(只兼容 Solidity 源码, 跑自定义 VM)。三年半后的 2026 年这套分类仍是行业共同语言: Taiko 还在追 Type 1、Scroll/Linea 卡在 Type 2、zkSync Era 是 Type 4 的代表——而 Polygon zkEVM 主网 beta 即将于 2026-07 sunset。zkEVM 是「把整个 EVM 状态机做成 ZK 电路」的工程, 让业务方写 Solidity 就能享受 ZK 安全性。Vitalik 2022-08 The different types of ZK-EVMs 把 zkEVM 沿「EVM 兼容性 ↔ prover 效率」轴分成 4 Type。

16.1 Type 1: 完全等价以太坊

「不改任何 Ethereum 系统组件来加快证明」。优点: 以太坊 L1 自身可以接入。缺点: prover 极慢(2024 年初约 16 分钟一个区块, 2026 年降到几十秒)。

代表项目:

- Taiko: 基于以太坊 validators 的 based rollup, 目标 Type 1;
- 以太坊 L1 enshrining zkEVM(2026 路线图): 执行层未来可能直接接 zkEVM。

16.2 Type 2: 完全 EVM 等价

EVM 字节码层等价, 但内部状态表示可改(如改 Merkle Patricia Trie 为更 ZK 友好的 Verkle Tree)。

代表项目:

- Scroll: 原 Type 3, 2024-2025 升到接近 Type 2;
- Linea(ConsenSys): 目标 Type 2。

16.3 Type 2.5: EVM 等价但调整 gas 表

为难以证明的操作(如 KECCAK256、MODEXP)涨 gas 价。中间形态。

16.4 Type 3: 几乎 EVM 等价

少数 precompile 不支持, 少数 opcode 行为略变。

代表项目:

- Kakarot zkEVM: Cairo 实现的 zkEVM, 目前 Type 3, 目标 Type 2.5;
- Polygon zkEVM(已退役): 曾是 Type 3。

16.5 Type 4: 仅源码兼容

不兼容 EVM 字节码, 但能编译 Solidity / Vyper 到自定义 VM。

代表项目:

- **zkSync Era**: 编译 Solidity 到 zkEVM bytecode(自定义);
- **Starknet**: 通过 Kakarot 提供 EVM 兼容, 但原生是 Cairo。

16.6 2026 年的重大变化

- Polygon zkEVM 主网 beta 在 2026-07-01 正式 sunset(提前 12 个月通知)。Polygon 把资源转向 AggLayer + Plonky3 + 跨链结算;
- 以太坊基金会公开提及「L1 enshrining zkEVM」: 意味着以太坊执行层可能在中期路线图直接接入 ZK proof 验证;
- **real-time proving** 实现: SP1 Hypercube + Plonky3 + S-two 三个系统都达到「proof 比区块时间快」的里程碑。

16.7 选 zkEVM 的工程考量

- 想要业务零改动: Type 1 / Type 2(Taiko / Scroll / Linea);
 - 想要最快 prover: Type 4(zkSync / Starknet);
 - 已有大量 Solidity 代码 + 在意 EVM 等价: Scroll、Linea;
 - 想要 ZK 原生体验: Starknet(Cairo)。
-

CHAPTER 18

第 17 章 zkVM 实测对比(2026-04 最新数据)

zkVM 性能竞赛的当代奇观：每周都有一家 在博客刷新 benchmark，每月都有一家说自己是「fastest」。SP1 Hypercube 2025-05 实测以太坊 block 22309250(143 笔交易、3200 万 gas)，16 张 RTX 5090 跑 10.8 秒——证明系统第一次跑得比一个 ETH slot(12 秒)还快。但所有数字都是 **self-reported**：Jolt 自报比 SP1 快 2×，但还是 alpha；Pico 自报世界最快但生态最窄。这一章给你一张 2026-04 的快照(明天就过时)，更重要的是教你如何带着怀疑去看 benchmark。

数据来源：a16z a16z/zkvm-benchmarks (2026-Q1 持续更新)+ Succinct succinctlabs/zkvm-perf + zkbenchmarks.com 社区面板 + Fenbushi 2025-06 横评(SP1/Risc0/OpenVM/Pico/ZKM/Jolt/Nexus/Novanet)+ 各家 release notes。

17.1 Fibonacci 1000 万 cycles

下表均为各项目 self-reported，未经第三方独立复现，横向对比优先看上述社区聚合源。

系统	配置	proof 时间	proof 大小	verifier 时间
Risc0 v3 (self-reported)	CPU baseline	19m10s	~300 KB	~10 ms
Risc0 v3 (self-reported)	GPU (RTX 4090)	~2m	~300 KB	~10 ms
SP1 v4 pre-Hypercube (self-reported)	AVX2 CPU	2m11s	~500 KB	~10 ms
SP1 v4 + Groth16 wrap (self-reported)	AVX2 + wrap	2m52s	~250 B	~3 ms
SP1 Hypercube (self-reported)	16× RTX5090	< 1s(推算)	~1 MB	~10 ms
Jolt 2026-02 alpha (self-reported)	CPU	~50s	~10 MB	~50 ms
Cairo + S-two 2.0 (self-reported)	CPU	~30s	~80 KB	~10 ms

注：Jolt 自报(self-reported)性能 2× SP1 / 5× Risc0 是基于 2026-02 alpha 的 preliminary，且尚未经过完整审计；正式版本对决待 Hypercube 之后再 做。

17.2 ETH L1 区块证明

2026 年最受关注的 benchmark：能否在 12 秒(一个 ETH slot)内证完一个区块。

系统	时间(block 22309250, 143 tx, 32M gas)	硬件
SP1 Hypercube(2025-05 实测)	10.8 秒	16× RTX 5090
SP1 Hypercube(2026-Q1)	99.7% 区块 < 12 秒	16× RTX 5090
Risc0 R0VM 2.0(2025-04 release)	44 秒	GPU 集群
Cairo + S-two	几十秒级	CPU

real-time proving 是「以太坊 L1 enshrining zkEVM」路线图的前置条件(\$16.6)。

17.3 zkVM 选型矩阵

场景	推荐	备选	不推荐
上 EVM 链验证(追求最小 gas)	SP1 + Groth16 wrap	Risc0 + Groth16 wrap	裸 STARK
已有 Rust 代码、想最快上手	Risc0	SP1	Cairo(要重写)
写 Cairo / 上 Starknet	Cairo + S-two	—	RISC-V 系
研究 / 实验 lookup-only zkVM	Jolt	—	生产用
客户端证明(手机 / 浏览器)	Binius64 + WHIR(实验)	SP1 client	Risc0(资源重)
长链增量计算	Lurk + Nova	Sangria + PLONKish	单层 STARK

CHAPTER 19

第 18 章 应用版图：从 zkRollup 到 zkOracle

讲到这里基础设施齐备了，但你回头看会发现：迄今为止讲的全是「肌肉」——多项式、配对、AIR、FRI、folding——这些都不是 ZK 的「为什么」。ZK 的「为什么」是 Worldcoin 用一颗 Orb 给 1800 万人做了「proof of human」、是印度国家身份 Aadhaar 用 Anon Aadhaar 把 RSA 签名压成 200 字节 proof、是 Aleo 让 Circle 把 USDC 的隐私版搬上链给机构 payroll 用。这章把肌肉对应到「生产中的脏活」：zkRollup 压缩 L2、zkBridge 替代多签、zkID 替代密码、zkML 替代「相信我跑了 GPT-2」——七个真实战场。

到此为止本模块讲完了 ZK 的「核心能力」(PCS + 证明系统 + 算术化 + zkVM)；下面把它们映射到真实生产场景。

18.1 应用全景图

ZK 在 Web3 的七大落地场景：

1. **zkRollup**：压缩 L2 交易、把状态转移正确性证给 L1；
2. **zkBridge**：跨链消息、light client 证明；
3. **zkID**：去中心化身份(人 / 文件 / 凭证)；
4. **Private DeFi**：隐私交易、机密余额、隐私 DEX；
5. **Proof of Solvency**：交易所偿付能力证明；
6. **zkML / opML**：链上验证 ML 推理；
7. **zkOracle / zkCoprocesor / zkEmail**：链外可信计算。

18.2 zkRollup 深度

主流 zkRollup 项目(2026-04)：

- **zkSync Era**(Matter Labs)：Type 4, PlonKup, 自定义 EraVM；TVL 数十亿；
- **Starknet**(StarkWare)：Cairo VM + S-two 2.0；2026-01 已切到 S-two 主网；
- **Linea**(ConsenSys)：Type 2 路线, Halo2 PSE fork；
- **Scroll**：Type 2, Halo2 PSE 起步, 2025+ 部分迁 Plonky3；
- **Taiko**：Type 1 based rollup；与 ETH validators 共享排序；
- **Polygon zkEVM**：2026-07 sunset；Polygon 转向 AggLayer + Plonky3。

18.3 zkBridge 与跨链证明

跨链桥靠 light client + ZK 证链 A 事件给链 B(详细谱系见 §32)。主流：Succinct(SP1 衍生 prover network)、Polyhedra zkBridge(多链 light client)、=nil; Proof Market、LayerZero ZK。

18.4 zkID：去中心化身份

项目	核心 ZK 机制	2026 状态
Worldcoin / World	Orb 扫虹膜 → IrisCode 哈希做唯一性去重；World ID 用 Semaphore 风格 action-scoped nullifier 防双花。注意：iris 本身不直接进 ZK 电路，nullifier 是按 (identity, action_id) 派生的，而不是从 iris 派生	18M 验证人类 (Orb 扫过虹膜) / 38M App 用户；450M proof 已生成；2026-04 与 Okta、Vercel 集成「proof of human」拦截 AI 流量
Anon Aadhaar	印度国家身份 RSA 签名 → ZK 证给链上	10 亿+ 人口的国家身份接入；PSE 维护
Sismo Connect	ZK Badge + Sign in 协议	隐私 SSO 替代 Sign in with Google
zkPassport	各国电子护照 (ICAO) DG 签名 → ZK	公民身份证明、机场 / 签证场景
Polygon ID / Iden3	通用 ZK credential 系统	企业级 KYC + 隐私

18.5 Private DeFi：隐私金融

项目	模型	2026 状态
Aztec Network	UTXO + Note + Noir 智能合约	主网 2026-Q1 上线；Noir 1.0 pre-release；私有 DeFi TVL 头部
Penumbra	Cosmos 系 L1, shielded staking + DEX	TVL 较小，但 IBC 兼容 + 全栈隐私
Railgun	以太坊主网上的 shielded pool	隐私 DEX + 借贷
Tornado Cash	已被合规风波冲击；新一代 Privacy Pools (Vitalik 2023 论文) 尝试「合规 + 隐私」	仍有 fork 在用
Aleo	snarkVM + L1 隐私链	2026-01/02 Circle USDCx、Paxos USAD 私有版上 Aleo
Nocturne / Umbra	隐私转账 / stealth address	中等热度

2026 年私有 DeFi TVL 1.5B+, Aztec + Railgun 主导。2026 趋势：「合规友好的隐私」——可证「这笔钱不来自 OFAC 制裁地址」+ 隐私的双重保证。

18.6 Proof of Solvency

证明交易所资产 $\geq$ 用户负债。FTX 后用户不再裸信任 CEX：

- Binance、Kraken、OKX 等用 Merkle Sum Tree + ZK proof (每个用户能验自己的余额被包含、总和正确)；
- 部分前沿方案 (zkPoR、Summa) 用更纯粹的 ZK 隐藏单用户余额；

- Vitalik 2022-11 写过一篇 Having a safe CEX: proof of solvency and beyond，是这一路线的奠基。

18.7 zkML 全景

zkML 解决: dApp 用 ML 模型决策时如何证明「推理结果来自指定 model X」。

主流 zkML 框架对比表：

框架	后端	模型规模上限(2026-Q1)	特色
EZKL(zkonduit)	Halo2 PSE + Icicle GPU	ResNet/Transformer 千万参数; CIFAR-10 模型分钟级	ONNX 直接喂; MSM GPU 加速后比 CPU 减 ~98%; proof ~几十 KB
Modulus Labs	Halo2 + Plonky2	~18M 参数 / 50 秒(强力 AWS 实例)	做过 Leela Chess Zero(链上 AI 国际象棋)+ RockyBot; Worldcoin / AI Arena 客户
Giza + Orion	Cairo + S-two	中等	Starknet 生态; LuminAIR 做 verifiable agents
ORA	zkML + opML 双轨	大模型走 opML	Onchain AI Oracle (OAO); 与 Arbitrum / Polygon 集成
OpenGradient	自建链 + EZKL 内嵌	通用	「链原生 AI 推理」
Lagrange zkPyTorch	Lagrange ZK Coprocessor	PyTorch 直接	2026 起 GPU 优化
Risc0 zkML	RISC-V STARK + Bonsai	模型规模极广但 prover 慢	RISC Zero v3 生态; Bonsai 远程证明
Bionetta	UltraGroth	极小 proof(320 B)	针对资源受限场景

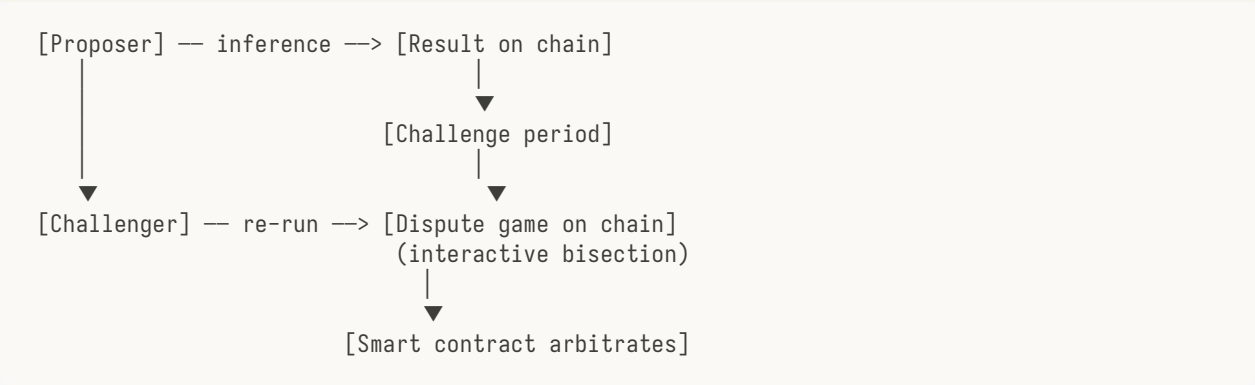
zkML 当前性能现实(2026-Q1): 千万参数 ResNet 在消费级 GPU 上 zkML 推理 + proof ~分钟级; 数十亿参数 LLM 直接 zkML 仍不可行(实践走 layer 片段证明 + opML); 所有主流框架在加 GPU 支持(CUDA / Icicle); EZKL 自报比 Risc0 快约 66×、比 Orion 快 ~3×。Lagrange DeepProve 2025 的 GPT-2 完整推理证明是关键里程碑(详见 §33)。

18.8 opML (Optimistic ML)

opML 模仿 Optimistic Rollup:

1. 链下跑推理, 链上提交 commitment;
2. 挑战期内任何人可以重跑;
3. 发现不一致就提交 fault proof(fraud proof);
4. 挑战期结束后结果终结。

架构示意：



主流实现：ORA opML(与 Arbitrum/Polygon 集成)、Modulus opML(部分场景)。

zkML vs opML 选型：

维度	zkML	opML
终结性	即时	挑战期(小时-天)
成本	高(prover 算力)	低(链下挑战者复算)
模型规模上限	千万参数(2026)	几乎不限(任意大模型)
信任假设	数学正确性	至少一个诚实挑战者
适用场景	资金敏感、不能等	公共物品、可以等

18.9 zkOracle / zkCoproprocessor

链下可信计算的派生：

- Brevis、Axiom：historical state lookup(「证明地址 A 在过去 100 万个区块的总转账 ≥ 1 ETH」)；
- =nil；Proof Market：proof 经济市场；
- Lagrange ZK Coprocessor：通用链下计算；
- RISC Zero Steel：Solidity 调用链下大型计算 $\rightarrow$ zk proof 回链上；R0VM 2.0 起把 ETH block proof 时间从 35 分钟降到 44 秒。

18.10 zkEmail

利用 DKIM 签名(邮件服务商对邮件头签)做 ZK 证明。ZK Email(Aayush Gupta 等)的核心声明形如「证明我有一封从 alice@google.com 来的邮件，但不暴露内容」，用例：zk-airdrop、链上身份恢复、whistleblower。早期版本曾被发现 under-constrained 漏洞(电子邮件地址欺骗)，是「ZK 应用层最容易出 bug」的代表案例。

CHAPTER 20

第 19 章 实战 1: Circom 2.2 + snarkjs 0.7 端到端

设想 Alice 的需求: 她的钱包给 Tornado Cash fork 存了 1 ETH, 现在想取出来到一个全新地址。她要向合约证明「我知道某个 secret, 使 $\text{Poseidon}(\text{secret})$ 等于这棵 Merkle 树里的某个叶子」——但不能告诉合约这个 secret 是什么, 否则别人就能撞库。这就是 ZK 隐私池的核心电路。本章把这个 Poseidon 原像证明从 0 跑到部署: Phase 1 ceremony → 编译 → 生成 witness → 出 proof → Solidity verifier 上 Anvil。30 分钟内你能在自家电脑上完成 Tornado Cash 之类项目用的同款流水线。

目标: 实现「我知道 x 使 $\text{Poseidon}(x) = y$ 」并把 verifier 部署到本地以太坊网络。完整代码在 `code/circom-poseidon/`。

19.1 环境准备

依赖: Node.js 22+、Rust 1.80+、circom 2.2.2、Foundry。

```
# 1. circom 用 Cargo 装 (不要用 npm 上的同名包)
git clone https://github.com/iden3/circom.git
cd circom
git checkout v2.2.2
cargo install --path circom

# 2. snarkjs (0.7.6 是 2026-01 release 的最新版本)
npm install -g snarkjs@0.7.6

# 3. circomlib (提供 Poseidon、MiMC、PedersenHash、MerkleProof 等组件)
mkdir poseidon-demo && cd poseidon-demo
npm init -y
npm install circomlib@2.0.5

# 4. Foundry (部署 Solidity verifier 到本地 Anvil)
curl -L https://foundry.paradigm.xyz | bash && foundryup
```


19.2 电路文件

```
pragma circom 2.2.2;

include "node_modules/circomlib/circuits/poseidon.circom";

template PoseidonPreimage(N) {
  signal input preimage[N];      // 私有输入: x (多个 field 元素)
  signal input expectedHash;     // 公开输入: y
  signal output ok;

  component h = Poseidon(N);
  for (var i = 0; i < N; i++) {
    h.inputs[i] <== preimage[i];
  }

  expectedHash == h.out;
  ok <== 1;
}

component main { public [expectedHash] } = PoseidonPreimage(2);
```

关键语法:

- `<==` : 赋值并约束(推荐);
- `<=` : 仅赋值不约束(**under-constrained** 头号源头, 慎用);
- `==` : 纯约束等号;
- `component main { public [...] }` : 必须显式列出公开信号。

19.3 完整流水线

```
# 编译电路: 得到 .r1cs / .wasm / .sym
circom poseidon_preimage.circom --r1cs --wasm --sym

# Phase 1: Powers of Tau
snarkjs powersoftau new bn128 12 pot12_0000.ptau -v
snarkjs powersoftau contribute pot12_0000.ptau pot12_0001.ptau \
  --name="First contribution" -v -e="some random text"
snarkjs powersoftau prepare phase2 pot12_0001.ptau pot12_final.ptau -v

# Phase 2: 电路相关 setup
snarkjs groth16 setup poseidon_preimage.r1cs pot12_final.ptau circuit_0000.zkey
snarkjs zkey contribute circuit_0000.zkey circuit_final.zkey \
  --name="Contributor" -v -e="another random text"
snarkjs zkey export verificationkey circuit_final.zkey verification_key.json

# 生成 witness
node poseidon_preimage_js/generate_witness.js \
  poseidon_preimage_js/poseidon_preimage.wasm input.json witness.wtns

# 生成 proof
snarkjs groth16 prove circuit_final.zkey witness.wtns proof.json public.json

# 链下校验
snarkjs groth16 verify verification_key.json public.json proof.json

# 导出 Solidity verifier
snarkjs zkey export solidityverifier circuit_final.zkey verifier.sol

# 用 Foundry 部署到 Anvil
anvil &
forge create verifier.sol:Groth16Verifier --rpc-url http://localhost:8545 \
  --private-key 0xac0974...80
```

19.4 生产里别这样做

- 不要自己跑 Phase 1: 直接用 Hermez Powers of Tau (powersOfTau28_hez_final_*.ptau) 或以太坊 KZG ceremony 输出;
- Phase 2 至少 5-10 个独立、不可勾结的贡献者;
- 不要相信「跑通了」: 跑通只代表诚实证明能验, under-constrained 几乎从来不会让诚实证明失败, 必须靠 circomspect、Picus、ZKAP 等工具或形式化验证发现;
- EIP-2098 注意: 早期的 Solidity verifier 模板有 malleability 风险, snarkjs 0.7.6 已默认修复。

CHAPTER 21

第 20 章 实战 2: Noir 1.0 等价实现 + 体验对比

写完 Circom 版本之后,把同一个 Poseidon 原像电路用 Noir 1.0 重写一遍——这是了解「ZK 工具链下一代是什么样」的最快路径。Aztec 自家协议电路 2024 起全部切到 Noir, 2026-02 发布 1.0 pre-release, 意味着语言级稳定性进入「发版前最后冲刺」。同样几行代码,错误提示从 Circom 的「constraint not satisfied at line 142」变成 Rust-like 的「expected Field, found u32」——这种“少几小时调试”的体验差,就是 Noir 在 2026 年成为新项目首选的原因。

20.1 Noir 是什么

Noir 是 Aztec 力推的 ZK DSL, 语法 Rust-like, 后端可换(默认 Barretenberg 走 PLONK; 也能输出到 Halo2、Plonky2)。2026-02 Noir 发布 1.0 pre-release, 意味着语言级稳定性进入「发版前最后冲刺」。Aztec 自家协议电路已全部用 Noir 重写。

20.2 等价 Poseidon 电路

```
// src/main.nr
use dep::std::hash::poseidon;

fn main(preimage: [Field; 2], expected_hash: pub Field) {
  let h = poseidon::bn254::hash_2(preimage);
  assert(h == expected_hash);
}
```

完整流程:

```
noirup                                # 装 nargo (Noir 工具链)
nargo new noir-poseidon
# 编辑 src/main.nr 和 Prover.toml
nargo check                          # 类型检查 + 生成 Prover.toml/Verifier.toml 模板
nargo execute                        # 跑 witness
nargo prove                          # 生成 proof
nargo verify                         # 链下验证
bb write_solidity_verifier           # 生成 Solidity 合约
```

20.3 Circom vs Noir 体验对比(2026-04 实测)

维度	Circom 2.2	Noir 1.0
学习曲线	陡(要先理解 R1CS / 约束语义)	缓(接近写 Rust)
生态 / 库	circomlib 老牌, 覆盖最广	std 还在补全, 但成长快
错误提示	编译器消息偏底层	编译器消息接近 rustc
后端灵活性	只接 R1CS(Groth16/PLONK)	ACIR 中间表示 + 多后端(Barretenberg/Halo2/Plonky2)
上链	直接 Solidity verifier	直接 Solidity verifier
主要风险	under-constrained 容易写出	当前 std 库部分 alpha; 编译器审计仍在进行
新项目推荐度	☆☆☆	☆☆☆☆

工程结论: 新项目优先 Noir(体验、错误提示、后端灵活性全面胜出); 老项目维护或极致最小 proof 继续 Circom(生态成熟); 学习路径: 先 Circom → 再 Noir。

CHAPTER 22

第 21 章 实战 3: SP1 / Risc0 zkVM 证明斐波那契

到这一章你已经手写过两个电路(Circom + Noir)——下一步是「不写电路」: 直接写 Rust, 让 zkVM 把 Rust → RISC-V → AIR 约束自动搞定。同样一段递归 `fib(20)` 在 SP1 与 Risc0 上能跑出几乎相同的代码——本章拿这个最小例子做对比, 让你体感「zkVM 工程师」与「电路工程师」的工作流差异。SP1 v5 Hypercube 已是 ETH L1 real-time proving 的标杆, Risc0 ROVM 2.0 把 ETH 区块证明从 35 分钟压到 44 秒——他们的差异不在最简单的 Fibonacci 上, 但工具链气质(host SDK / 远程证明 / 预编译)已经能感受到。

21.1 SP1 版本

完整代码在 `code/sp1-fib/`。核心两文件:

```
// program/src/main.rs (guest 端, 编译成 RISC-V)
#![no_main]
sp1_zkvm::entrypoint!(main);

pub fn main() {
    let n = sp1_zkvm::io::read::<u32>();
    let mut a: u64 = 0;
    let mut b: u64 = 1;
    for _ in 0..n {
        let c = a.wrapping_add(b);
        a = b;
        b = c;
    }
    sp1_zkvm::io::commit(&n);
    sp1_zkvm::io::commit(&a);
}
```

```
// script/src/main.rs (host 端, 跑 prover)
use sp1_sdk::{ProverClient, SP1Stdin, include_elf};

const ELF: &[u8] = include_elf!("fibonacci-program");

fn main() {
    let client = ProverClient::from_env();
    let (pk, vk) = client.setup(ELF);

    let mut stdin = SP1Stdin::new();
    stdin.write(&20u32);

    let proof = client.prove(&pk, &stdin).run().unwrap();
    client.verify(&proof, &vk).expect("verification failed");
    println!("OK, fib(20)={}", proof.public_values.read::<u64>());
}
```

跑通:

```
cd code/sp1-fib
cargo prove build --bin fibonacci-program -p program
cargo run --release -p script
```

链上验证(可选):

```
SP1_PROVER=network cargo run --release -p script -- --groth16
```

得到约 250 字节的 EVM 友好 proof。

21.2 Risc0 等价版本

代码在 `code/risc0-fib/`。流程:

```
cargo install cargo-risczero
cargo risczero new fib --guest-name fib_guest
cd fib
cargo run --release
```

guest 代码(`methods/guest/src/bin/fib_guest.rs`)和 host 代码(`host/src/main.rs`)几乎与 SP1 版本同构。

21.3 SP1 vs Risc0 体感对比(2026-04)

维度	SP1 v5(Hypercube)	Risc0 v3.0.5
性能	Hypercube 后领先; real-time ETH proving	紧追, v3.0.5 起多项优化; ROVM 2.0 在路上
预编译	EVM 必备项齐全(Keccak/SHA256/secp256k1/ ed25519/BN254/BLS12-381)	全套 + RSA + pairing 路线图明确
形式化验证	已与 EF + Nethermind 完成 RISC-V 约束验证	ROVM 2.0 用 Picus 检查 determinism
Bonsai / 远程 prover	SP1 Network(远程证明服务)	Bonsai(GPU 集群即服务)
推荐场景	上以太坊链上验、需要 EVM precompile	通用 RISC-V 计算 + 多 host 部署

CHAPTER 23

第 22 章 练习

USENIX Security 2024 的 SoK 论文给了一个让人不安的数字：真实 ZK 漏洞中 $\approx 96\%$ 来自 **under-constrained**——电路看起来跑通了、proof 生成了、verifier 接受了，但其实有一条约束漏写了，恶意 prover 可以伪造任意陈述。这一章的 4 道题不是用来「考试」的，是用来给你装一个「闻 under-constrained 味道」的鼻子。每道题做完，你都会比 80% 的 ZK 简历强——因为那 80% 的人没真的找过漏洞。

起始代码在 `exercises/` 下，每题独立一个目录。

22.1 练习 1: Merkle Proof + Poseidon (Circom)

要求：写一个电路，证明「我知道一个叶子 leaf 和一条长度 20 的 Merkle path (兄弟节点 + 0/1 方向位)，使得叶子 hash 到 root」。

提示：

- 用 `circomlib/poseidon.circom` 的 Poseidon(2) 哈希 `left||right`;
- 方向位要约束 `dir * (1-dir) == 0`;
- root 公开, leaf 私有。

参考骨架在 `exercises/01-merkle-poseidon/`。这是 Tornado Cash、Semaphore、几乎所有 zk-airdrop 的基础原语。

22.2 练习 2: 找 under-constrained 漏洞

`exercises/02-under-constrained/buggy.circom` 提供一个看似正常的 IsZero 电路(缺约束)。

任务：

1. 找出哪一行让恶意 prover 可以伪造 `out = 0` 但 `in  $\neq$  0` (或反过来);
2. 给出 fix;
3. 用 `circomspect / picus` (在练习目录的 README 里有一键脚本) 扫描确认 fix 后无告警;
4. 在 `notes.md` 写下 30-100 字解释为什么 $\leftarrow$ 是危险源。

22.3 练习 3: Risc0 证明排序

写一个 guest 程序：读入 $n + n$ 个 u32，把它们排序后输出，并 commit「输入是输出的一个置换 + 输出非降序」这一陈述。host 端 verify 后打印 sorted。

提示：commit 输入和输出的 sha256 (或 Risc0 keccak precompile) 即可。完整骨架在 `exercises/03-risc0-sort/`。

22.4 练习 4 (进阶): Noir 实现 ECDSA 签名验证

证明「我知道一个 secp256k1 私钥 sk，用它签了 message m，得到 (r, s)」，但不暴露 sk。

提示: Noir std 已包含 `std::ecdsa_secp256k1::verify_signature` ; 公开输入 (m, pubkey), 私有输入 sk + (r, s)。

CHAPTER 24

第 23 章 AI × ZK: zkML、opML、AI 辅助 ZK 工程的边界

2025 年好几次出现一种营销话术:「让 LLM 帮你写 ZK 电路」。这听起来很美——但 USENIX Security 2024 的 96% under-constrained 事实告诉我们: LLM 写的电路 90% 概率有漏洞而它们都能跑通。AI 在 ZK 这条最后一英里的安全战场上, 目前能做的事比能不能做的事更值得讲清楚。这一章不重复 §18 的 zkML / opML 选型, 只讲两件人们容易搞错的事: (1) AI 反向辅助 ZK 工程能做到哪里? (2) 哪些事情你绝对不能让 AI 替你做?

zkML / opML 的定义、性能现实、选型表已在 §18.7 / §18.8 给出, 本章不重复, 只补两个角度: AI 反向辅助 ZK 工程的现状 (§23.3); AI 在 ZK 上的红线 (§23.4)。

23.3 AI 反向辅助 ZK 工程

到 2026 年这一领域早期但成型, 分四个方向:

1. 电路调试: LLM 解释 `circumspect` 的告警、提示「`←` 应改 `⇐`」之类。Trail of Bits、zkSecurity 在试验 LLM-augmented ZK auditing;
2. 论文阅读: Justin Thaler 第 4 章 `sumcheck` 的 PDF 喂给 Claude / GPT, 配合「对照课程视频第 8 讲讲一遍」效果不错;
3. 从规格生成电路骨架: 还远不到生产级——LLM 写的 Circom 电路 90% 概率有 under-constrained 漏洞, 因为 LLM 训练语料里 ZK 代码本就少, 正/反样本比极不健康;
4. 形式化验证辅助: Picus、ACL2 这类工具的 spec 往往要人手写, 有团队尝试用 LLM 把规格自然语言→spec, 初步效果可用但需要人复核。

23.4 AI 在 ZK 上的红线

ZK 是安全的最后一英里, 上面跑数十亿美元的协议不止一个。AI 局限请刻在脑子里:

1. AI 写 ZK 电路不可靠。under-constrained 不会让代码 panic、不会让测试失败、不会让 proof 验不过——但能让有恶意的 prover 伪造任意陈述。LLM 的幻觉在普通业务代码里能容忍, 在 ZK 里直接灾难;
 2. 不要让 LLM 替你做 `trusted setup`。Powers of Tau ceremony 的安全性来自人和人之间的不可勾结假设, 「我让 AI 帮我贡献一份」等于自废;
 3. Fiat-Shamir 误用不肉眼可见。LLM 写的 prover/verifier 哪怕长得正常, 少 hash 一项 transcript 就是 frozen heart 类灾难;
 4. 形式化验证仍然是金标准。SP1 + EF + Nethermind 把 RISC-V 约束做完全形式化验证、Risc0 ROVM 2.0 用 Picus 检查 determinism——AI 在这条路上是辅助不是主角。
-

CHAPTER 25

第 24 章 进一步阅读

24.1 推荐学习路线(从浅到深)

1. Vitalik 三连发(zkSNARK 入门, 2016-2017);
 2. Jordi Baylina 的 Circom 官方文档 + PSE 的 zk-mooc.github.io (视频 + lab);
 3. Justin Thaler Proofs, Arguments, and Zero-Knowledge (2023);
 4. ZK Whiteboard Sessions (zkHack): 每集 30 分钟讲一个具体协议;
 5. 0xPARC [zk-bug-tracker](#): 看真实漏洞清单, 比读论文长记性;
 6. SP1 / Risc0 / Halo2 官方 book + examples;
 7. SoK: What Don't We Know? Understanding Security Vulnerabilities in SNARKs (Chaliasos et al., USENIX Security 2024);
 8. Trail of Bits ZK audit 文章合集 (Axiom Halo2 deep dive 2025-05、circomspect 工具);
 9. WHIR / Binius / S-two 的源码 + 论文——前沿三剑客;
 10. 个人挑战: 把 0xPARC 的 ZKHack puzzle 全做一遍。
-

CHAPTER 26

第 25 章 参考资料(含 URL)

25.1 经典论文 / 教科书

- Vitalik 三连发 QAP: <https://vitalik.eth.limo/general/2016/12/10/qap.html>
- Vitalik Exploring Elliptic Curve Pairings: https://vitalik.eth.limo/general/2017/01/14/exploring_ecp.html
- Vitalik zk-SNARKs Under the Hood: https://vitalik.eth.limo/general/2017/02/01/zk_snarks.html
- Justin Thaler Proofs, Arguments, and Zero-Knowledge: <https://people.cs.georgetown.edu/jthaler/ProofsArgsAndZK.html>
- KZG10 原论文: <https://cacr.uwaterloo.ca/techreports/2010/cacr2010-10.pdf>
- Groth16 论文: <https://eprint.iacr.org/2016/260>
- PLONK 论文: <https://eprint.iacr.org/2019/953>
- Marlin 论文: <https://eprint.iacr.org/2019/1047>
- Bulletproofs 论文: <https://eprint.iacr.org/2017/1066>
- Spartan 论文: <https://eprint.iacr.org/2019/550>
- Brakedown 论文: <https://eprint.iacr.org/2021/1043>
- WHIR 论文: <https://eprint.iacr.org/2024/1586>
- Nova 折叠论文: <https://eprint.iacr.org/2021/370>
- USENIX Security 2024 SoK on SNARK vulns: <https://www.usenix.org/system/files/usenixsecurity24-chaliasos.pdf>

25.2 规范 / 文档

- Circom 文档: <https://docs.circom.io/>
- snarkjs 仓库: <https://github.com/iden3/snarkjs>
- Halo2 Book(Zcash): <https://zcash.github.io/halo2/>
- Halo2 PSE fork: <https://github.com/privacy-scaling-explorations/halo2>
- Plonky3 仓库: <https://github.com/Plonky3/Plonky3>
- SP1 仓库: <https://github.com/succinctlabs/sp1>
- Risc0 仓库: <https://github.com/risc0/risc0>
- Jolt 仓库: <https://github.com/a16z/jolt>
- Cairo + S-two 仓库: <https://github.com/starkware-libs/stwo>
- Noir 文档: <https://noir-lang.org/docs/>
- ETH KZG ceremony specs: <https://github.com/ethereum/kzg-ceremony-specs>
- Vitalik zkEVM 分类原文: <https://vitalik.eth.limo/general/2022/08/04/zkevm.html>

25.3 2026 年关键公告

- Plonky3 production-ready(Polygon): <https://polygon.technology/blog/polygon-plonky3-the-next-generation-of-zk-proving-systems-is-production-ready>

- SP1 Hypercube real-time proving: <https://blog.succinct.xyz/real-time-proving-16-gpus/>
- Risc0 形式化验证之路: <https://risczero.com/blog/RISCZero-formally-verified-zkvm>
- Noir 1.0 pre-release: <https://aztec.network/blog/the-future-of-zk-development-is-here-announcing-the-noir-1-0-pre-release>
- S-two 上 Starknet 主网: <https://www.starknet.io/blog/s-two-is-live-on-starknet-mainnet-the-fastest-prover-for-a-more-private-future/>
- Polygon zkEVM 2026 sunset 公告: <https://forum.polygon.technology/t/sunsetting-polygon-zkevm-mainnet-beta-in-2026/21020>
- Aleo 2026 进展: <https://aleo.org/>
- Worldcoin / World ID 2026 升级: <https://world.org/blog/announcements/at-last-trust-in-the-age-of-ai>
- Irreducible Binius64: <https://www.irreducible.com/posts/announcing-binius64>

25.4 安全资源

- 0xPARC zk-bug-tracker: <https://github.com/0xPARC/zk-bug-tracker>
- circomspect(Trail of Bits): <https://github.com/trailofbits/circomspect>
- Picus(Veridise): <https://veridise.com/picus/>
- ZKAP: <https://github.com/whyliuanqi/ZKAP>
- Trail of Bits Axiom Halo2 deep dive 2025-05: <https://blog.trailofbits.com/2025/05/30/a-deep-dive-into-axioms-halo2-circuits/>
- Aztec Connect 多重花费披露: <https://hackmd.io/@aztec-network/disclosure-of-recent-vulnerabilities>

25.5 zkML 资源

- EZKL 仓库: <https://github.com/zkonduit/ezkl>
- EZKL benchmarks: <https://blog.ezkl.xyz/post/benchmarks/>
- Modulus Labs Make ZKML Real: <https://medium.com/@ModulusLabs/chapter-8-make-zkml-real-a3a355b2b756>
- ORA opML 仓库: <https://github.com/ora-io/opml>
- The Definitive Guide to ZKML 2025: <https://blog.icme.io/the-definitive-guide-to-zkml-2025/>
- Worldcoin awesome-zkml: <https://github.com/worldcoin/awesome-zkml>
- Giza Hub: <https://github.com/gizatechxyz/Giza-Hub>

25.6 视频 / 课程

- ZK-MOOC(UC Berkeley): <https://zk-learning.org/>
- ZK Whiteboard Sessions(zkHack): <https://zkhack.dev/whiteboard/>
- Justin Thaler 课程视频: YouTube 搜 "Justin Thaler ZK"。

25.7 社区与会议

- ZKProof 标准化倡议: <https://zkproof.org/>
- ZK Summit(年度): <https://www.zksummit.com/>
- ETHGlobal ZK 系列 hackathon。

CHAPTER 27

第 26 章 隐私计算的兄弟姐妹：FHE、TEE、MPC、Garbled Circuits

ZK 不是隐私的全部答案——它是「证明 + 不泄密」一件事。如果你想让别人直接在密文上做计算而不解密？那是 FHE；想在 Intel 芯片的「黑盒小屋」里跑代码，连 OS 都看不到？那是 TEE；想 5 个人合作算一道题，谁也不知别人输入？那是 MPC；想把整张电路混淆成一次性的乱码？那是 Garbled Circuits。Vitalik 2020 写过 A Quick Garbled Circuits Primer——这一章把 ZK 放回它该在的位置：与 FHE / TEE / MPC / GC 并列的乐高积木之一，而不是隐私的全部答案。

ZK 解决「我证明给你看且不泄密」，但完整的隐私 / 机密计算栈还包含 FHE(在密文上算)、TEE(在硬件保护区里算)、MPC(多方协作算且谁也不知全貌)、Garbled Circuits(一次性混淆电路)。这些技术经常互相组合而非互相替代。

26.1 全同态加密 FHE(Fully Homomorphic Encryption)

FHE: 直接在密文上做加减乘，解密后等于在明文上做同样运算。

主流方案：

方案	适合的数据	代表实现
TFHE / TFHE-rs	布尔 / 整数(位级别精确)	Zama tfhe-rs(Rust)、concrete
CKKS	浮点 / 近似(ML 推理)	Microsoft SEAL、OpenFHE、HEAAN
BFV / BGV	整数	OpenFHE、PALISADE

FHE vs ZK 的核心对比：

维度	ZK	FHE
解决问题	证明「我做对了」+ 不泄密	让别人在我的密文上算 + 别人也看不到结果
计算位置	prover(持私)做完, verifier 只验	计算方完全不知数据
性能开销	prover 慢、verifier 快	比明文慢 $10^3 - 10^6$ 倍(2026 仍是)
链上典型用法	隐私转账、L2 状态压缩	机密智能合约(链上算密文)
是否需要私钥	prover 持 witness	计算方不持密钥；解密方持

26.2 Web3 的 FHE 三巨头(2026-04)

- **Zama(fhEVM + TFHE-rs)**：以太坊系最完整的 FHE 栈。fhEVM 让 Solidity 直接在密文上写合约(euint32、ebool 等加密类型)；TFHE-rs 是底层 Rust 库；Concrete 是 Python → FHE 的编译器。\$ZAMA token 2026-01 上线，路线图是从「20 TPS 今天 → 100,000+ TPS 配 ASIC」；

- **Fhenix**(融资 \$22M)：第一个把 FHE 做成 L2 落地的项目，CoFHE coprocessor 已部署到 Arbitrum；与 Zama 战略合作复用 TFHE-rs 内核；
- **Inco Network**(融资 \$4.5M)：「confidentiality-as-a-service」，把 FHE 当中间件给现有链用；混合 TEE + FHE+MPC，为速度和隐私做权衡。

26.3 ZK + FHE 的天作之合

2026 年最有意思的方向是「ZK + FHE 联姻」：

- 场景：dApp 用 FHE 算密文投票结果，再让计算方出 ZK proof 证「正确执行了协议」；
- 代表：Inco confidentiality-as-a-service 同时挂 ZK + TEE 两条 fallback；
- 未来：ZK 负责「正确性 + witness 隐藏」，FHE 负责「外包计算输入隐藏」——两者是隐私链的完整答案。

26.4 TEE：硬件 enclave 路线

CPU/GPU 厂商提供的隔离区(连 OS 也看不到)，在内部运行代码并出 attestation 证明「这段代码在真实 TEE 里执行过」。

主流硬件：

- Intel SGX(旧) / TDX(新, VM 级)；
- AMD SEV / SEV-SNP；
- NVIDIA H100 / H200 Confidential Computing (2024+ 给 GPU 加 TEE)；
- ARM CCA(移动端)；
- AWS Nitro Enclaves(云厂商封装)。

TEE vs ZK 对比：

维度	ZK	TEE
信任根	数学(离散对数 / 哈希)	硬件厂商 + 攻击者没物理拆芯片
性能开销	慢(prover 几秒几十秒)	几乎与明文同速
历史侧信道漏洞	少(数学层)	频繁：SGAxe、Foreshadow、ÆPIC、Plundervolt 等
2024 重大事故	—	TEE 漏洞致 Phala / Secret / Crust / IntegriTEE 等多链紧急升级；Oasis 因架构隔离未受影响
适合场景	资金敏感 / 强对抗	大模型推理、AI agent、对延迟极敏感的场景

26.5 Web3 的 TEE 主流玩家(2026-04)

- **Phala Network**: 「Confidential Cloud」, 对接 Intel TDX、SGX、AMD SEV、NVIDIA H100/H200; 产品线 Phala Cloud + Confidential VM + 私有 LLM 推理;
- **Oasis Sapphire / ROFL**: 「Trustless AWS」。ROFL Mainnet 已于 2025-07 上线, 让开发者把重计算放到 enclave, 结果再写回 Sapphire EVM 链; 推出 WT3 trustless DeFi trading agent;
- **Marlin**: 通用 TEE 计算市场;
- **TEN Protocol**(前身 Obscuro): 以太坊兼容机密 L2, TEE 内执行;
- **Secret Network**(最早做 TEE 隐私链, 受 SGX 漏洞影响多次升级);
- **Automata Network**: TEE 标准化与 attestation; 与 Flashbots、Marlin、Oasis、Phala、Secret、Poetic、Rena、SpaceComputer、t1、TEN 联合发起 TEE Wiki(2025-)。

26.6 MPC: 多方安全计算

n 方各持秘密 x_i , 协作计算 $f(x_1, \dots, x_n)$, 各方不知他人 x_i 。

经典 1980s 协议族:

- **Yao Garbled Circuits**(两方): 一方加密整张电路 → 另一方按 OT 解密对应输入 → 算出输出; Vitalik 2020 写过著名入门 A Quick Garbled Circuits Primer;
- **GMW / BMR**: n 方协议;
- **SPDZ / MASCOT / Overdrive**: 恶意安全 MPC;
- **Shamir Secret Sharing**: 门限解密 / threshold 签名(也算 MPC 子集)。

MPC vs ZK:

维度	ZK	MPC
输入主体	单方(prover)	多方
隐藏	隐藏 witness	隐藏每方输入
计算方	prover 自己	n 方协作
通信	1 次(非交互)	多轮(每个门可能要交互)
主流 Web3 用例	proof 单方提交	threshold 钱包、隐私拍卖、DKG(分布式密钥生成)

Web3 里 MPC 的常见落地:

- **MPC 钱包**: Fireblocks、ZenGo、Coinbase WaaS——把私钥分片到 n 方, 做 threshold ECDSA / Schnorr 签名;
- **Threshold BLS**: Drand、Aggregator network;
- **DKG(分布式密钥生成)**: Filecoin、ETH validator key 派生;
- **隐私拍卖 / 暗池**: COTI、Penumbra ZSwap 之类用 MPC 或 MPC + ZK 组合。

26.7 Garbled Circuits 在 Web3 的 2026 复兴

2024-2026 因 COTI V2 把 Garbled Circuits + ZK 做成 EVM 兼容机密 L2, 重回工程视野。

特点：

- 不需要 **trusted setup**；
- 轻量：声称比 ZK-only 方案快 $\sim 10\times$ 、proof 比 ZK 小 $\sim 10\times$ ；
- 限制：本质两方协议；多方场景要套 BMR；非常适合「单用户 + 计算服务方」的二人场景。

26.8 ORAM(Oblivious RAM)

ORAM 解决一个相关但不同的问题：即使加密了存储，访问模式(哪个地址被读了)也会泄露信息。例如你每次买 ETH 都访问内存第 7 槽，攻击者就能猜你在交易 ETH。

ORAM 让访问模式也不可区分。Web3 里常见用法：

- 隐私 DEX 的 **order book**：暗池防止前后跑(front-running)；
- 隐私 RAM 模型 zkVM：Risc0 / SP1 内部把内存访问做 ORAM-friendly 编码(实际是 Memory Argument, 不是严格 ORAM 但思想接近)；
- 私有数据库查询：MPC + ORAM 在医疗 / 金融数据共享。

26.9 这些技术的「正确组合姿势」

最常见的工程组合：

目标	推荐组合
隐私转账 / Mixer	ZK(Tornado/Aztec/Railgun)
链上私有 DeFi(合规友好)	ZK + Privacy Pools 选择性披露
链上算密文(投票、拍卖、AI)	FHE(fhEVM)+ 可选 ZK 验正确性
大模型推理 / AI agent	TEE(Phala / Oasis ROFL)+ 可选 ZK attestation
MPC 钱包 / threshold sig	MPC(Fireblocks / ZenGo)+ 可选链上 BLS 验证
跨链桥	ZK light client(zkBridge) / TEE bridge / MPC 多签
暗池 DEX	MPC / Garbled Circuits + ORAM

ZK 是这套乐高里的关键一块，单打独斗解决不了所有场景；FHE / TEE / MPC / GC 各自的边界要单独记。

26.10 互相对比口诀

ZK : 「你信我的数学」 → 单方 + 强信任根 + 慢
 FHE : 「我把密文给你算」 → 外包计算 + 极慢但越来越可行
 TEE : 「你信硬件 + 厂商」 → 快但受侧信道漏洞威胁
 MPC : 「我们一起算谁也不告诉谁」 → 多方协作 + 通信开销大
 GC : 「我把电路混淆好了，你按 OT 解开」 → 两方协议 + 单次性
 ORAM : 「连我访问哪儿你都不知道」 → 隐藏访问模式

CHAPTER 28

第 27 章 折叠方案展开: Nova / SuperNova / HyperNova / ProtoStar / Sangria

类比一句话讲完折叠是干什么的: 信用卡用了一个月, 每天单独算利息要算 30 次; 但银行只在月底结算一次。Nova 给 ZK 这个领域打开了同样的逻辑: 把 1000 步计算的 1000 个 R1CS 实例 折成 一个 relaxed R1CS, 1000 步过程中根本不做 SNARK, 最后才一次性 SNARK。Filecoin 的 SDR 文件 PoS 证明、Lurk Lisp zkVM、用户手机端隐私转账增量证明——这些"长链"应用之所以能跑起来, 靠的就是折叠。

第 13 章给了折叠方案的速览。本章展开——因为 2024-2026 折叠是 zkVM 加速、客户端证明、增量计算的核心。

27.1 为什么需要折叠

经典递归 SNARK 是「proof 套 proof」: layer N 的 proof 验证 layer N-1 的 proof, 每层都要做完整 SNARK 验证(KZG 配对、IPA 内积、FRI query)。这每层都贵。

折叠(folding)的核心 insight: 先不做 SNARK 验证, 把多个 instance 累积(fold)成一个 instance, 最后只 SNARK 一次。

类比: 信用卡月底一次性结清, 平时不算。

27.2 Nova(Kothapalli-Setty-Tzialla 2022)

- 输入: 两个 R1CS 实例 (U_1, W_1) 和 (U_2, W_2) ;
- 输出: 一个 Relaxed R1CS 实例 $U + W$, 使得它「等价于」前两个实例都被满足;
- 折叠: 几次 MSM, 不做任何 SNARK;
- 最后一次 Spartan-style SNARK 证 final relaxed R1CS;
- 限制: 所有 instance 必须走同一个电路。

适用场景: 长链增量计算(IVC, Incremental Verifiable Computation), 如 Filecoin 的 SDR(Stacked DRG)文件 PoS 证明、长链 ZK rollup state transition。

27.3 SuperNova(Kothapalli-Setty 2022)

把 Nova 推广到多电路(NIVC): 每步可选 N 个不同电路之一, 天然对应「每步执行不同 RISC-V opcode」。

27.4 HyperNova(Kothapalli-Setty 2023)

把 Nova 移植到 multilinear + sumcheck 体系, prover 复杂度更低, 适合 Spartan/Jolt/HyperPlonk 系; 2024-2026 折叠主流。

27.5 ProtoStar (Bünz-Chen 2023)

把 Nova/SuperNova/HyperNova 核心思想抽象成 sumcheck-based predicate; 给定任意 PLONKish 电路自动生成对应 folding scheme, 工程友好。

27.6 Sangria (Mohnblatt 2023)

把 PLONKish 风格电路(自定义门 + lookup)做成折叠。被 Aztec、Geometry 等团队评估用作 PLONK 系递归。

27.7 ProtoGalaxy (Eagen-Gabizon 2023)

ProtoStar 轻量版:「一对一 fold」→「多对一 fold」, 理论更高效, 工程正在落地。

27.8 Mangrove / KiloNova / NeutronNova / 2025 新折叠

- Mangrove(2024): 递归折叠 + lookup;
- KiloNova / NeutronNova(2024-2025): 进一步压缩 prover 内存;
- 2026 的趋势是「折叠 + Lasso lookup + GPU 友好」三件套合体。

27.9 折叠 vs 普通递归 vs 累积方案

方案	中间步骤代价	最后一步	代表
普通递归 SNARK	每步全 SNARK 验	同	Cycle-of-curves (PLONK over 2 曲线)
累积方案 (accumulator)	每步轻 deferred 验	一次 IPA	Halo2
折叠(folding)	每步几次 MSM, 不做 SNARK	一次 Spartan 类	Nova / HyperNova

经验: 短链(< 10 步) → 普通递归够用; 中等链(10-1000 步) → 累积; 长链(> 1000 步) → 折叠。

27.10 谁在用折叠

- Lurk(Argument): Nova / HyperNova 做底层;
- LambdaClass Whirlaway: 实验性 folding + WHIR;
- Risc Zero / SP1 / Pico 内部部分阶段;
- Filecoin SDR proofs: Nova IVC;
- 私有付款 IVC: 在用户手机上每笔交易增量证明。

CHAPTER 29

第 28 章 Mina + o1js: 客户端零知识 webapp

具体场景：你想做一个 web 应用让用户在自己浏览器里证明「我有有效的 Google 账号」而不上传 cookie。以太坊范式做不到——业务逻辑必须发到链上重跑。Mina 把「业务逻辑在客户端跑 + 链只验 proof」做成原生范式，整条链恒定 22 KB。

28.1 Mina 是什么

Mina 是一条「整条链 22 KB」的 ZK-native L1：每个区块的状态转移都被压成一个递归 SNARK，新节点同步只需要下载一个 proof。

核心组件：

- Kimchi: 自家的 PLONK 衍生(Berkeley upgrade 后启用)，大量 lookup + custom gate;
- Pickles: 递归层, proof of proof of proof, 最终把整条链折成 22 KB;
- o1js(前身 SnarkyJS): TypeScript 框架, 在浏览器/Node.js 直接写 zkApp。

28.2 zkApps 的特点

zkApp 在用户浏览器本地执行合约逻辑并生成 proof, 将 proof + 状态发到链上; 链只验 proof, 不重跑业务逻辑。计算移客户端, 链做最小共识——与以太坊范式根本不同。

28.3 o1js 速览

```
import { Field, SmartContract, state, State, method } from 'o1js';

class Square extends SmartContract {
  @state(Field) num = State<Field>();

  init() {
    super.init();
    this.num.set(Field(3));
  }

  @method async update(square: Field) {
    const currentState = this.num.getAndRequireEquals();
    square.assertEquals(currentState.mul(currentState));
    this.num.set(square);
  }
}
```

亮点：TypeScript 写电路、浏览器跑 prover；ZkProgram 支持递归 proof(Pickles)；@state 自动加密；NPM 包 bundler 友好。

28.4 客户端零知识的工程意义

「所有人都能在自己设备上证明」打开一系列以太坊范式做不到的应用：

- 隐私默认：用户的输入永远不上传；
- 抗审查：链上看不到业务逻辑细节；
- GDPR 友好：数据从不出端；
- 零信任 web2 集成：Google / Apple 账号在浏览器内零知识证明有效性。

28.5 Mina 性能与现状(2026-Q1)

- 链大小恒定 ~22 KB——任何设备都能成全节点；
- TPS 较低(几个 / 秒到几十 / 秒)，但对客户端零知识场景够用；
- 2024 起 Berkeley 升级把核心 prover 切到 Kimchi, prover 速度提升 ~5 倍；
- o1Labs 持续补强 o1js 工具链(prover key caching 等性能优化)。

28.6 何时选 Mina

- 想做浏览器内零知识 webapp；
- 用户体验优先(不要让用户跑命令行)；
- 业务逻辑相对简单(复杂电路在客户端会卡)；
- 强 web2 接入 / 身份场景。

不要选：高 TPS 的 DeFi、复杂多人状态机(那是以太坊 / Solana 的菜)。

CHAPTER 30

第 29 章 Lurk: 内容寻址的 Lisp zkVM

设想 Filecoin 上有一段链下程序, 它处理了几 GB 的私有数据, 输出一个结果。你怎么向链上合约证明「这个结果确实是 hash X 对应的程序, 对 hash Y 对应的输入跑出来的」? 传统 zkVM 想做这件事得把整段程序 + 输入打包进 proof, 太重。Lurk 用一种古老的语言哲学解决问题: Lisp 的「代码即数据」+ Poseidon 哈希做内容寻址 = 程序和数据都是 hash, 跑完链上只需验「(hash_program, hash_input, hash_output)」这个三元组的关系。它本身就是给 IPFS / Filecoin 量身定做的 zkVM。

29.1 Lurk 是什么

Lurk (Lambda, the Ultimate Recursive Knowledge) 是 Filecoin / Protocol Labs 的 Chhi'mèd Künzang 设计的 静态作用域 Lisp dialect + 内容寻址 + 递归 zk-SNARK。

设计哲学 (Common Lisp + Scheme): 代码即数据 (homoiconic)、Poseidon 哈希构建复合数据实现内容寻址、与 IPFS/IPLD 集成、整个执行轨迹可被递归 SNARK 证明。

29.2 演化路线

- Lurk α / β (2022-2024): 基于 Nova (椭圆曲线折叠);
- Lurk 0.5 (2024): 换骨架到 STARK + Memoset + Plonky3, 性能跨数量级提升;
- Sphinx (Argument Computer, 2024 开源): SP1 fork, 内部嵌 Lurk 的 STARK + Memoset 设计; 用于 Wormhole light client、Kadena 等;
- 2025-2026: 维护方从 Lurk Lab 演化为 Argument (GitHub 组织 `argumentcomputer`), 持续推进 STARK 引擎。

29.3 内容寻址 + zk 的工程含义

Lurk 数据 = (hash, content), commit = 公布 Poseidon hash。链上合约可证「某 hash 对应程序跑了 N 步输出 X」或「IPFS CID 对应代码输出 Y」——信任从「相信你跑过」升级到「hash 可验」。

29.4 应用方向

- Filecoin FVM: 链下智能合约 + 链上验证;
- IPFS / IPLD 数据完整性: CID 内容证明;
- 跨链 light client: Sphinx 给 Wormhole 做的就是这一类;
- 递归 SNARK 实验: Lurk 一直是 Nova 派系最成熟的工程实现之一。

29.5 何时关心 Lurk

- 你在 Filecoin / IPFS 生态做事;
- 你需要「内容寻址 + 可证明计算」组合;
- 你想看 Lisp + ZK 怎么结合 (这本身是有趣的研究方向)。

CHAPTER 31

第 30 章 Aztec / Penumbra / Aleo: 隐私链主网状态盘点
(2026-04)

2026 年初的三件大新闻一起改写了「隐私链」的定义: (1) Aztec Network 2026-03-31 上线 Alpha 主网——以太坊第一个支持完全私有智能合约的 L2; (2) Aleo 2026-01 拿下 Circle USDCx, 2026-02 又拿下 Paxos USAD, 机构级隐私稳定币第一次成型; (3) Penumbra 把 Cosmos 系隐私 DEX 跑了三年, 2026 起把 IBC 桥到 50M+ shielded TVL。三条链各自有不同的隐私哲学(Aztec 靠 UTXO + Note + Noir、Penumbra 把 IBC 隐私化、Aleo 把记录加密给 KYC 留口子)——选哪条做你的隐私 dApp, 得看你想隐私什么。

第 18 章给了一行项目对比。要做隐私 dApp 必须知道的最新信息: 每条链的主网状态、技术栈、当前限制、bug bounty。

30.1 Aztec Network: 「以太坊的私有 L2」

主网状态:

- Alpha mainnet 2026-03-31 上线——Ethereum 第一个支持完全私有智能合约的 L2;
- 当前 ~1 TPS, ~6 秒出块;
- 存在 critical 已知漏洞, 团队官方提示「只存能承受损失的金额」;
- Beta 准入门槛: > 10 TPS、99.9% uptime、3 个月内无 critical bug 报告;
- v5 release(2026-07 计划)会打包修复。

技术栈:

- 语言: Noir(1.0 pre-release, Rust-like);
- 后端: Barretenberg(Aztec 自家 PLONK + UltraPlonk 衍生);
- 模型: UTXO + Note(类似 Zcash Sapling), 账户隐私默认;
- 智能合约: 私有合约(用户客户端跑 prover)+ 公共合约(rollup 共识层跑);
- bug bounty 总额: \$2,000,000(Immunefi 主托管)。

适合: 私密支付、机密 DeFi、合规+隐私双满足的早期实验。不适合: 高 TPS DeFi(Beta 前勿上)、关键金融基础设施(v5 前勿上)。

30.2 Penumbra: Cosmos 系的全栈隐私

主网状态:

- 2023 年上主网, 是 IBC 兼容的隐私枢纽 L1;
- TVL ~3.77M(2025-11 数据)——比 Aztec / Railgun 小, 但 IBC 接入面广;
- 2026 预测: Cosmos 隐私扩张到 ~50M shielded TVL;
- 协议层「Shielded Upgradability」机制让链升级也保持隐私连续性。

技术栈:

- 语言: Rust + 自家 zk DSL;
- 证明系统: Groth16 + 自家 PLONK 衍生;
- 模型: UTXO + Note + zk-DEX;
- 全栈隐私: 转账、staking、治理、DEX 全部 shielded;
- DEX 关键设计: **batched swap intents**——prevent intra-block front-running; swap 输出永不公开, 直接 mint 到用户的 shielded pool。

适合: Cosmos/IBC 隐私应用、隐私 DEX、shielded staking/governance。不适合: 以太坊生态(桥代价高)、超高频交易(batched swap 有延迟)。

30.3 Aleo: ZK-native L1 + 机构隐私稳定币**主网状态:**

- 2024-09 上主网;
- 2026-Q1 重大事件:
- Circle USDCx(2026-01): Circle 在 Aleo 主网上线 USDC 的隐私版, 瞄准机构需求;
- Paxos USAD(2026-02): Paxos 上线机密稳定币 USAD, 瞄准企业 payroll + treasury;
- Stablecore 集成(2026-02): 1,600 家银行接入 USDCx / USAD;
- Toku payroll(2026 中商用): 第一个机密稳定币 payroll 解决方案;
- 网络性能: snarkOS v4 升级后交易确认快 5×。

技术栈:

- 语言: Leo(自研, 类 Rust);
- 证明系统: Marlin + 自家递归(基于 BLS12-377 + BW6-761 cycle);
- VM: snarkVM / snarkOS;
- 模型: UTXO + Records; 记录里加密发件人地址(仅收件人可解密密钥), 便于 KYC + 隐私平衡。

适合: 机构隐私稳定币、合规私有 DeFi、企业 payroll/treasury。不适合: 以太坊兼容场景(Aleo 独立 L1)、需 EVM 工具链(Leo 新语言)。

30.4 三条链对比表

维度	Aztec	Penumbra	Aleo
主网时间	2026-03 Alpha	2023	2024-09
形式	Ethereum L2	Cosmos L1	独立 ZK-native L1
语言	Noir	Rust + 自家 DSL	Leo
证明系统	UltraPlonk (Barretenberg)	Groth16 + PLONK	Marlin + 递归
隐私模型	UTXO + Note	UTXO + Note	UTXO + Records
标杆生态	DeFi 隐私实验	IBC 隐私枢纽	机构稳定币 (Circle/Paxos)
当前阶段	Alpha(小心进)	主网稳定	主网稳定 + 机构入场
推荐场景	以太坊生态隐私早期	Cosmos 生态隐私	合规机构隐私

CHAPTER 32

第 31 章 zk 协处理器: Brevis / Axiom / Herodotus / Lagrange / RISC Zero Steel / SP1 Reth

具体例子: 你要给 DEX 做一个「用户过去一年累计交易量 ≥ 10 ETH 给空投」的合约。以太坊合约只能访问当前状态和最近 256 个区块头, 过去一年的几百万个区块数据没法在 EVM 里读。zk 协处理器 = 把链下大计算 + 历史查询包成 zk proof, 链上一次验证拿结果。

31.2 主流 zk 协处理器(2026-04)

项目	主架构	主战场
Brevis	ZK 数据协处理器 + 通用 zkVM(Pico)	历史数据 + L1 RTP; 2026-Q1 实测在 64×RTX5090 集群上 45M gas 区块平均 6.9 秒、P99 < 10s
Axiom	Halo2 PSE 自研	可验证查询 + 电路回调
Herodotus	多链 storage proof	跨链历史状态访问
Lagrange	ZK + Optimistic 混合(State Committees)	跨链计算性能 + 与 Polyhedra 合作做 OP rollup 快速 finality
RISC Zero Steel	Risc0 zkVM 提供的 EVM view 接口	让 Solidity 调链下大计算回 ZK proof; R0VM 2.0 把 ETH block proof 时间从 35 分钟压到 44 秒
SP1 Reth	Succinct 把 Reth(执行客户端)跑在 SP1 内	出 ETH L1 区块 zk proof; real-time proving 的标杆

31.3 协处理器 vs zkRollup 区别

- zkRollup: 完整 L2 状态机; 状态根写回 L1; 用户在 L2 交互;
- 协处理器: 没有自己状态; 只把「链下计算 + 历史数据查询」做成 ZK proof; 用户在 L1 调合约时拿到结果;
- zkRollup 是「另一条链的镜像」, 协处理器是「L1 合约的私人秘书」。

31.4 典型用例

- DeFi: 根据用户长历史 ETH 持仓 / 交易量做积分奖励(Brevis 主战场);
- 空投验证: 「证明这个地址在 2020-2026 持仓 ≥ 1 ETH 至少 365 天」;
- 跨链状态查询: 「Polygon 上 X 余额是多少」(Herodotus 强项);
- 可验证 AI / ML: 把模型推理放协处理器;
- L1 RTP(Real-time Proving): 把整个 ETH 区块塞协处理器证明, 是「以太坊 enshrining zkEVM」路线图的前置。

31.5 EF zkEVM Tracker 路线图 (2026-Q1)

主要参与者: SP1 Turbo (Succinct)、Pico (Brevis)、RISC Zero、ZisK、Airbender (zkSync 2026-01 公布)、OpenVM (Axiom)、Jolt (a16z)。所有项目都在向「< 12 秒证一个 ETH L1 区块」这个共同目标冲刺。

31.6 zkVM benchmark 大对决 (2026-Q2 数据)

来源: Brevis zkvm-bench、Succinct zkvm-perf、Fenbushi 2025-06 横评、Pico-GPU 公告 2025-06。

zkVM	CPU 性能 vs 最优	GPU 性能 vs SP1	关键工程
Pico (Brevis)	1.7X-2.55X SP1 (CPU 自报世界最快)	单卡 RTX 4090 上比 SP1 快 25%	模块化 zkVM; 与 Brevis 数据协处理器同栈
SP1 Hypercube	仅次于 Pico; 但生态最广	baseline; 16X RTX5090 99.7% ETH block <12s	EF + Nethermind 形式化验证 RISC-V; 预编译最齐
RISC Zero v3	中等	紧追 SP1	ROVM 2.0 上 ETH block 35 min → 44s; Steel + Bonsai 生态
OpenVM (Axiom)	中等	中等	模块化 + 自定义指令集
Airbender (zkSync)	2026-01 公布	强调 RTP	RISC-V zkVM 的 zkSync 自家实现
Jolt	仍 alpha	部分场景比 SP1 快 2X / RISC0 快 5X	完全靠 Lasso lookup; Twist/Shout 进一步优化
ZKM	中等	中等	MIPS zkVM

2026 年 zkVM 性能竞争是「周维度」更新——任何静态 benchmark 表都会过时。生产选型时务必看: (1) 最近 4 周的官方 benchmark; (2) 是否经过审计; (3) 与你已有栈的兼容性。

31.7 选型建议

- 历史 ETH 数据查询 → Brevis / Axiom / Herodotus;
- 跨链 + OP rollup 快速 finality → Lagrange + Polyhedra zkBridge;
- 想在 Solidity 里调链下大算 → RISC Zero Steel / SP1 Reth;
- 自家 rollup 集成 → 选与你 zkVM 同栈的 (SP1 → SP1 Reth; Risc0 → Steel; Plonky3 → Pico / Brevis)。

CHAPTER 33

第 32 章 zkBridge 谱系展开

跨链桥被盗的累计金额在 2022-2024 年突破 \$20 亿——Ronin \$625M、Wormhole \$326M、Nomad \$190M、BNB \$570M——每一次被盗，背后都是同一个故事：「m-of-n 多签人不勾结」这个信任假设被打破。zkBridge 是这套信任假设的根本性升级：不再靠人，靠数学。链 A 出区块头 → ZK 证「这个头确实是链 A 共识层验过的」→ 链 B 验 proof。Polyhedra zkBridge 已支持 25+ 链、Succinct Telepathy 接入 LayerZero、Lagrange State Committees 给 OP rollup 提供快速 finality——本章给主流 zkBridge 项目的 2026-04 状态盘点。

32.1 zkBridge 解决什么

Light client + ZK 把链 A 区块头 / 事件存在性证给链 B。多签桥(Ronin \$625M、Wormhole \$326M、Nomad \$190M) 累计被盗 \$20 亿+，是行业转向 ZK 桥的根本动因——信任假设从「m-of-n 多签人不勾结」降到「数学正确性 + 1 个诚实 prover」。

32.2 主流 zkBridge 项目(2026-04)

24.2.1 Polyhedra zkBridge

- 思路：zkLightClient 验整条 ETH PoS 共识 + 多链 state transition；
- 已支持 25+ 条链；
- 与 Lagrange State Committees 合作做 OP Rollup 快速 finality：State Committee 由 EigenLayer 重质押的 ETH validators 组成，对每个 OP rollup 状态转移签 BLS attestation，再用 ZK 把 attestation 压成 proof；
- 2026 路线图：与 Caldera 合作把 zkBridge 接到 AppRollups。

24.2.2 Succinct Telepathy

- Succinct Labs 出品；
- 用 SP1 zkVM 写 light client logic；
- 已被 LayerZero、Across 等接入。

24.2.3 Lagrange Labs

- 提供 State Committees + zkProver Network 两层：
- State Committees：EigenLayer 重质押验证器签 attestation → 可即时 finalize OP rollup；
- zkProver Network：分布式 prover 市场；
- 与 Polyhedra 合作覆盖 OP rollup 快速 finality 场景；
- DeepProve(zkML) 也是 Lagrange 同公司另一条产品线。

24.2.4 LayerZero ZK

- LayerZero 的 ZK 增强分支；
- 把 ULN(Ultra Light Node)配置升级到 ZK light client。

24.2.5 =nil; Foundation Proof Market

- 通用 proof 经济市场；
- 任何链可以挂订单：「我要证某条链的某状态」，市场里 prover 接单；
- 不只是桥，也是协处理器 + zkOracle 的基础设施。

32.3 zkBridge 信任假设升级表

桥类型	信任根	历史损失
多签桥	m-of-n 多签人	Ronin \$625M、Wormhole \$326M、Nomad \$190M、BNB \$570M 等
Optimistic 桥	至少 1 个诚实挑战者 + 挑战期	Nomad 是混合，挑战期没起作用
zkBridge	数学正确性 + 至少 1 个诚实 prover	极少 (2026 仍未发现重大盗币)

32.4 选型

- 跨 EVM 链 + 想要终极信任：Polyhedra zkBridge / Succinct Telepathy；
- OP rollup 快速 finality：Lagrange + Polyhedra；
- LayerZero 生态升级：LayerZero ZK；
- 自定义 proof / 经济市场：=nil; Proof Market。

CHAPTER 34

第 33 章 zkML 实测大对决(2026-04)

「证明这张图是 ResNet-50 在我的输入上跑出来的」这句话在 2023 年是 Modulus Labs 一万个 GPU 小时的赌局；2025 年 Lagrange DeepProve 把同样的事情做到分钟级；2025 年下半年 DeepProve-1 第一次把整个 GPT-2 (1.5B 参数) 的 forward pass 完整 zk-proven——zkML 从「玩具」跨进「工业级 LLM 可证明推理」这一里程碑就发生在你写这段话前 18 个月。但「能做」不等于「便宜」：千万参数 ResNet 上消费级 GPU 仍是分钟级、数十亿参数 LLM 仍走 layer 片段证明 + opML 的混合路线。本章给 2026-04 时点的实测数据 + 性能突破时间线。

第 18.7 节给了框架对比；本章给 benchmark 数据 + 性能突破时间线。

33.1 关键里程碑

- 2023: Modulus Labs 在 18M 参数模型上做出第一批分钟级 zkML proof；
- 2024: EZKL 接入 Icicle GPU, MSM 加速 ~98%；
- 2025: Lagrange 发布 DeepProve: 声称比 EZKL 快 158× 生成 proof、671× 验证；
- 2025: DeepProve-1 — 第一个对完整 GPT-2 推理生成 ZK proof 的工业级系统；
- 2026-Q1: 每个主流 zkML 框架都已或正在加 GPU 支持。

33.2 zkML 框架性能对比(2026-Q1)

框架	proof 时间(CNN 264k)	相对 EZKL 加速	proof 大小	模型规模上限
Lagrange DeepProve	~1-5 秒(视模型)	158× faster proving	中等	已证完整 GPT-2
EZKL	几十秒到分钟	1× baseline	几十 KB	千万参数
Modulus Labs	50 秒(18M 参数 / AWS)	~ EZKL 同级	几十 KB	~18M 参数
Giza Orion	中等	无公开对比	几十 KB	中等
Risc0 zkML	慢(通用 zkVM)	EZKL 自报快 ~66×	较大	模型灵活但 prover 慢
OpenGradient	自建链内嵌 EZKL	同 EZKL 级	同	中等
Bionetta	极小 proof (UltraGroth)	不重 prover 速度	320 字节	资源受限场景
TensorPlonk	理论 1000× 加速(实验)	Daniel Kang 实验	中等	实验

33.3 DeepProve 的工程意义

DeepProve-1 是第一个把 LLM 完整推理塞进 ZK 的工业级系统：

- 模型：OpenAI GPT-2 (~1.5B 参数)；
- 整个 forward pass 都被 zk-proven；
- 标志 zkML 从 CNN/小模型进入 LLM 阶段；
- 「证明 LLM 输出来自特定模型」从「不可能」变「成本高但可行」。

33.4 zkML / opML / TEE-ML 三选一

维度	zkML	opML	TEE-ML
终结性	即时	挑战期	即时
成本	高	低	低
模型规模	LLM 已可 (DeepProve) 但贵	任意	任意
信任根	数学	至少 1 个诚实挑战者	硬件 + 厂商
侧信道风险	极低	极低	中等 (SGX 历史漏洞)
适用	资金敏感 + 即时	公共物品 + 可等	AI agent + 性能优先

代表项目：

- zkML：EZKL、Modulus、Giza、ORA zkML、Lagrange DeepProve；
- opML：ORA opML、Modulus opML (部分场景)；
- TEE-ML：Phala、Oasis ROFL、Marlin、TEN Protocol。

33.5 zkML 性能瓶颈分析

zkML 慢的根本原因：①神经网络以矩阵乘为主，每个乘法在 ZK 电路中付代价；②ZK 电路用整数/定点数，与浮点推理结果存在量化差异。

2026 年的工程突破方向：

- GPU 加速 (Icicle / TensorPlonk / Lagrange)；
 - lookup-friendly 神经网络 (用查找表代替 sigmoid / GELU 计算)；
 - Sumcheck / multilinear 路线 (DeepProve、Jolt-style zkML)；
 - opML / TEE-ML 互补：大模型走非 zkML，小关键判定走 zkML。
-

CHAPTER 35

第 34 章 Brillig: Noir 的「不约束」逃生通道

具体场景：你要在 Noir 里证「我知道 x 使 $x^2 == y$ 」。算开方在电路里要数千约束，但只验 $y^2 == x$ 只要一条乘法约束。Brillig 让你在 prover 端写普通 Rust 算 sqrt，链上电路只验。

34.1 ACIR vs Brillig 的双轨编译

Noir 编译产出两种字节码：

- **ACIR**: Abstract Circuit Intermediate Representation——约束系统中间表示，每条 opcode 对应电路里的一条约束；
- **Brillig**: **unconstrained** 字节码虚拟机，只算不约束。

双轨是为了「算贵验便宜」类计算——除了 sqrt，还有解大模数倒数、大表线性扫描查找等。

34.2 unconstrained 关键字

```
// 在 Noir 里:
unconstrained fn approx_sqrt(x: Field) → Field {
    // 这里写的代码会编译成 Brillig 字节码，跑在 prover 端，不进电路
    // ...任意 Rust 风格代码 (含循环、赋值、分支)
}

fn main(x: pub Field, y: Field) {
    // 1. 在 Brillig 里算 (便宜)
    let approx = approx_sqrt(x);

    // 2. 在 ACIR 里验 (约束)：用一条乘法约束就能验对
    assert(approx * approx == x);
}
```

34.3 Brillig 的危险

`unconstrained` 是有意的逃生通道，用错则是 under-constrained 漏洞的最大产地：

-  在 `unconstrained` 里算结果但忘了在外面 `assert`：prover 可以返回任意值；
-  在 `unconstrained` 里做安全敏感判断（「这个用户是否 KYC 过」）但 `assert` 不严格；
-  正确用法：`unconstrained` 算「困难」，`assert` 验「容易」，且 `assert` 必须语义等价。

34.4 Brillig 的工程意义

Brillig 是 Noir 「易用 + 高性能」的关键设计——让常见的「prover 多算 / verifier 少算」模式有语言级支持。Circom 没有等价机制（要手写 $\leftarrow$ 然后小心补约束），所以 Noir 在这点上更工程友好。

34.5 实战 checklist

- 任何 `unconstrained` 函数的返回值，必须在调用方加 `assert`；
-

assert 关系必须语义完全等价于 unconstrained 里的计算；

- ☐ Code review 时把 unconstrained 当 unsafe：每个出现都要审；
 - ☐ 用 Noir profiler 检查约束成本，避免不该 unconstrained 的也 unconstrained 了。
-

CHAPTER 36

第 35 章 ZK 工程师 2026 路线图

2026 年招聘市场上「ZK Circuit Engineer」是 crypto 最稀缺的岗位，没有之一。Trail of Bits、Veridise、zkSecurity、Cantina——四家头部审计公司常年在挂招人，给的钱也是 crypto 圈最高的几档。但「难招」不是因为工程师懒，而是因为 ZK 这条路有 6 个不同的能力级别(L0 入门到 L5 安全研究)，每升一级都需要至少 6 个月。这一章是路线图——12 周的学习地图、6 级能力天花板、5 类岗位画像，给你看清楚自己现在在哪一格、下一格要去哪里。

35.1 学习地图

Week 1-2 : 概念 + Vitalik 三连发 + 本模块第 1-3 章
 Week 3-4 : 跑通 Circom Poseidon + Noir 等价 (第 19-20 章)
 Week 5-6 : Justin Thaler 教科书第 1-7 章 + ZK-MOOC 视频
 Week 7-8 : 跑通 SP1 / Risc0 Fibonacci + 找漏洞练习 (第 21-22 章)
 Week 9-10 : 选一个证明系统精读 (Halo2 PSE / Plonky3)
 Week 11-12: 做一个端到端项目: zk-airdrop / proof-of-solvency / zkML demo

35.2 工程师能力级别

级别	标志
L0 入门	知道 SNARK / STARK 区别; 能跑 snarkjs demo
L1 应用	写得出 Circom / Noir 电路; 做 hashlock + Merkle proof
L2 工程	选系统、调 prover、部署 verifier、看懂 audit 报告
L3 深入	写 Halo2 PSE 自定义门; 分析 under-constrained; 理解 Fiat-Shamir
L4 研究	设计自己的算术化 / PCS; 推动 STARK / Folding 边界
L5 安全	能用 Picus / circomspect 找漏洞、能形式化验证

35.3 行业职位 2026

- ZK Circuit Engineer(最稀缺): Circom / Halo2 / Noir 电路设计 + 审计;
- zkVM Engineer: SP1 / Risc0 / Cairo / Pico 内部贡献;
- ZK Application Engineer: 在 zkRollup / 隐私 dApp 上做产品;
- ZK Auditor: Trail of Bits / Veridise / zkSecurity / Cantina;
- ZK Researcher: 学院 + 工业界联合; 写论文 + 发版本;
- ZK Infra: 协处理器、桥、prover network。

35.4 不会过时的能力

- 密码学严谨性: 把每个声明的可靠性 / 零知识性写清楚;
- 形式化验证思维: 用 SMT / 定理证明工具证明性质;

- Rust 工程: 现代 ZK 生态 ~80% 是 Rust;
 - 审视假设: trusted setup、Fiat-Shamir、信任根、域选择——每条假设的代价你都要能讲清楚;
 - 跨密码学栈思维: ZK + FHE + TEE + MPC 的组合能力。
-

CHAPTER 37

第 36 章 ZK 路线图与开放问题(2026-2030)

ZK 是密码学领域过去十年最确定的赌局之一——但不是所有问题都解决了。后量子 SNARK 上链、zkML LLM 成本、客户端证明大众化、ASIC 时代二元域 vs 小素数域之争——这些问题决定了 2030 年的 ZK 长什么样。这一章把 2016-2026 已完成的里程碑串成时间线，再列出 7 个未解的开放问题——这是给「已经入门」的人看的路线图，告诉你接下来 5 年最值得押注的方向在哪里。

36.1 已完成的里程碑

- 2016: Groth16, 工业级最小 SNARK;
- 2018: STARK 论文 + 抗量子;
- 2019: PLONK, universal setup;
- 2020: Halo2, IPA + 累积方案;
- 2021: Brakedown, linear-time prover;
- 2022: Nova 折叠; Plonky2; Vitalik zkEVM 分类;
- 2023: 以太坊 KZG ceremony 完成(14 万人); UltraPlonk 主流化;
- 2024: Plonky3 alpha; Risc0 v1 主流; SP1 测试网; EZKL GPU; DeepProve;
- 2025: SP1 Hypercube; S-two; Plonky3 production; Risc0 R0VM 2.0; DeepProve-1 GPT-2; Aztec 主网倒计时; Pico; ROFL Mainnet;
- 2026-Q1: S-two 上 Starknet 主网; Aztec Alpha 主网; Aleo 机构稳定币(Circle/Paxos); Plonky3 production-ready 公告; real-time proving 多家达成。

36.2 仍然开放的问题

- 真正完全形式化验证的端到端 zk stack——SP1 / Risc0 还在路上;
- 后量子 SNARK 上链——所有 EVM 主流 SNARK 仍依赖椭圆曲线 / 配对, 量子计算成熟后必须迁移;
- zkML 上 LLM 的成本——DeepProve-1 是突破, 但 prover 仍贵;
- Trusted setup 完全消除: 透明 setup(Halo2 IPA、STARK、Brakedown、Binius、WHIR)成熟到能完全替换 KZG 的那一天;
- ASIC 时代的二元域 / 小素数域 / Mersenne-31 三选一: 硬件和算法的赛跑;
- 客户端证明大众化: 手机 / 浏览器跑 prover 几秒内完成(Mina / Aztec / Binius64 都在追);
- 隐私 + 合规的折中工程化: Privacy Pools / 选择性披露 / OFAC 兼容性。

36.3 5 年视角的猜想

- 以太坊 L1 enshrining zkEVM(执行层直接接 ZK proof);
- 主流隐私链合并机构稳定币 + KYC 友好选择性披露;
- zkVM 性能跟上 EVM 解释器原生速度(Hypercube / Pico 已经 closing);
- LLM 推理上链(zkML + opML + TEE-ML 混合)成为基础设施;
- 折叠 + Lasso + GPU 三件套成为新 zkVM 默认;
- AI 辅助 ZK 工程进入「LLM 给 spec、人审 + 形式化验证」流水线(仍是辅助而非主角)。

CHAPTER 38

第 37 章 Powers of Tau 后续 ceremony 与 Trusted Setup 治理

14 万人——这是 2023 年初 Ethereum KZG ceremony 的参与人数，也是密码学历史上参与人数最多的 trusted setup。任何一个里面只要一个人销毁了自己的 τ_i ，整套 SRS 就是安全的 (1-of-N 信任假设)。两年后的今天，每个 Solidity 工程师写 PLONK 系电路时其实都在「免费蹭用」这 14 万人累积的安全性——这是 ZK 工程的低估资产之一。本章讲 ETH KZG 之后还会有什么 ceremony、Perpetual Powers of Tau 怎么持续追加贡献、a16z 提出的「On-Chain Trusted Setup Ceremony」想法，以及为什么「自己跑 ceremony 给主网用」几乎永远是错的。

37.1 ETH KZG ceremony 之后呢

以太坊 KZG ceremony(2023-Q1, ~14 万人参与，目前规模最大的 trusted setup)，输出 SRS 用于：

- EIP-4844 blob commitment(已上线)；
- Full Danksharding(路线图)；
- Verkle Tree(执行层未来状态承诺)；
- 任何 PLONK / KZG-based SNARK 的 universal SRS(社区共享)。

37.2 后续 ceremony 计划

- Danksharding ceremony(计划于 2025-2026 启动)：为 Full Danksharding 做更大规模 SRS(参数 N 更大)，仍是 1-of- N 信任假设；
- Perpetual Powers of Tau(社区维护，从 2019 起)：80+ 轮持续追加贡献，文件名形如 powersOfTau28_hez_final_*.ptau；circom 教程默认下载这一套；
- Aztec Ignition v2(如果再做)：因 Aztec 切到 UltraPlonk + 复用 ETH KZG，已不再需要自己 ceremony；
- Filecoin / Aleo / 其他链：各自仍维护独立 phase 2 ceremony(电路相关)。

37.3 Trusted Setup 治理的工程教训

- 复用比新做安全：14 万人参与的 ETH KZG SRS 任何项目都该优先复用；
 - transcript 一定要发布：所有 ceremony 必须公开 transcript + 验证脚本，让社区独立校验；
 - 新参与机制：a16z 在 2024 提出 [On-Chain Trusted Setup Ceremony](#)——把 ceremony 本身搬到链上验证，进一步降低 sequencer 信任；
 - Transparent setup 是终极目标：Halo2 IPA / STARK / Brakedown / Binius / WHIR 均不需 setup。
-

CHAPTER 39

第 38 章 增量参考资料(25-37 章)

38.1 FHE / TEE / MPC

- Zama fhEVM: <https://github.com/zama-ai/fhevm>
- Zama TFHE-rs: <https://github.com/zama-ai/tfhe-rs>
- Zama Master Plan: <https://www.zama.org/post/zama-fhe-master-plan>
- Fhenix: <https://www.fhenix.io/>
- Inco Network: <https://www.inco.org/>
- Phala Confidential AI: <https://phala.com/>
- Oasis ROFL: <https://docs.oasis.io/build/rofl/>
- Oasis ROFL Mainnet 公告 (2025-07): <https://www.the-blockchain.com/2025/07/02/oasis-protocol-foundation-launches-rofl-mainnet-verifiable-offchain-compute-framework-powering-ai-applications/>
- Marlin: <https://www.marlin.org/>
- TEN Protocol: <https://ten.xyz/>
- TEE Wiki (Automata 等多方): <https://forum.teekettle.org/>
- Vitalik Garbled Circuits Primer: <https://vitalik.eth.limo/general/2020/03/21/garbled.html>
- David Evans A Pragmatic Introduction to Secure MPC: <https://securecomputation.org/>
- COTI V2 Garbled Circuits 文档: <https://docs.coti.io/>
- Zellic MPC From Scratch: <https://www.zellic.io/blog/mpc-from-scratch/>

38.2 折叠系列

- Nova 论文: <https://eprint.iacr.org/2021/370>
- SuperNova 论文: <https://eprint.iacr.org/2022/1758>
- HyperNova 论文: <https://eprint.iacr.org/2023/573>
- ProtoStar 论文: <https://eprint.iacr.org/2023/620>
- Sangria (Mohnblatt): <https://geometry.xyz/notebook/sangria-a-folding-scheme-for-plonk>
- ProtoGalaxy 论文: <https://eprint.iacr.org/2023/1106>

38.3 Mina / Lurk / Brillig

- Mina o1js docs: <https://docs.minaprotocol.com/zkapps/o1js>
- Mina Kimchi: <https://minaprotocol.com/blog/kimchi-the-latest-update-to-minas-proof-system>
- Mina Pickles 递归: <https://docs.minaprotocol.com/zkapps/o1js/recursion>
- Lurk 文档 / 论文: <https://research.protocol.ai/publications/lurk-lambda-the-ultimate-recursive-knowledge/amin2023.pdf>
- Argument (Lurk 维护方): <https://argument.xyz/>
- Sphinx 开源公告: <https://argument.xyz/blog/sphinx-oss/>
- Brillig 文档: <https://noir-lang.org/docs/noir/concepts/unconstrained>

- Noir 编译器原理: <https://medium.com/distributed-lab/noir-under-the-hood-from-code-to-constraints-b3af7a54f00c>
- jtriley Noir Circuit Backend: <https://jtriley.substack.com/p/noirs-circuit-backend>

38.4 隐私链主网状态

- Aztec Alpha mainnet 公告: <https://aztec.network/blog/announcing-the-alpha-network>
- Aztec Road to Mainnet: <https://aztec.network/blog/road-to-mainnet>
- Penumbra: <https://penumbra.zone/>
- Penumbra DEX 文档: <https://guide.penumbra.zone/dex>
- Penumbra Shielded Upgradability: <https://www.penumbra.zone/blog/shielded-upgradability>
- Aleo 主网 Circle USDCx: <https://aleo.org/post/aleo-circle-launch-of-usdcx/>
- Aleo Paxos USAD: <https://www.theblock.co/post/389101/privacy-preserving-usad-stablecoin-launches-aleo-layer-1-mainnet-paxos-partnership>

38.5 zk 协处理器 / zkVM benchmark

- Brevis 文档: <https://coprocessor-docs.brevis.network/>
- Pico GPU benchmark: <https://blog.brevis.network/2025/06/27/announcing-pico-gpu-setting-a-new-zkvm-benchmark-with-gpu-acceleration/>
- Brevis zkVM-bench: <https://github.com/brevis-network/zkvm-bench>
- Babybear-labs benchmark: <https://github.com/babybear-labs/benchmark>
- a16z zkvm-benchmarks: <https://github.com/a16z/zkvm-benchmarks>
- Succinct zkvm-perf: <https://github.com/succinctlabs/zkvm-perf>
- Axiom: <https://www.axiom.xyz/>
- Herodotus: <https://herodotus.dev/>
- Lagrange: <https://lagrange.dev/>
- RISC Zero Steel + Bonsai: <https://dev.risczero.com/litepaper>
- ZKsync Airbender: <https://blockeden.xyz/blog/2026/01/30/zksync-airbender-fastest-risc-v-zkvm-ethereum-proving/>

38.6 zkBridge

- Polyhedra zkBridge: <https://www.zkbridge.com/>
- Polyhedra 2026 路线图: <https://blog.polyhedra.network/polyhedra-moving-into-2026/>
- Polyhedra X Lagrange State Committees: <https://medium.com/@lagrangelabs/announcing-polyhedra-networks-lagrange-labs-partnership-zkbridge-fast-finality-for-rollups-with-364323194b65>
- Succinct Telepathy: <https://blog.succinct.xyz/sp1-testnet/>
- =nil; Proof Market: <https://proof.market/>

38.7 zkML 进阶

- Lagrange DeepProve 公告: <https://lagrange.dev/blog/announcing-deepprove-zkml>
- DeepProve-1 GPT-2: <https://lagrange.dev/blog/deepprove-1>
- DeepProve 仓库: <https://github.com/Lagrange-Labs/deep-prove>

- TensorPlonk(Daniel Kang): <https://medium.com/@danieldkang/tensorplonk-a-gpu-for-zkml-delivering-1-000x-speedups-d1ab0ad27e1c>
- Worldcoin awesome-zkml: <https://github.com/worldcoin/awesome-zkml>
- zkML Singularity 2025 Comprehensive Analysis: <https://academy.extropy.io/pages/articles/zkml-singularity.html>

38.8 折叠 / Lookup 学术

- Lasso 论文: <https://people.cs.georgetown.edu/jthaler/Lasso-paper.pdf>
- Twist/Shout(Jolt 内部): <https://github.com/a16z/jolt/blob/master/EngineeringOverview.md>
- Sum-Check over small fields(Justin Thaler 2024): <https://people.cs.georgetown.edu/jthaler/small-sumcheck.pdf>
- HyperPlonk 加速(zkSpeed 2025): <https://eprint.iacr.org/2025/620.pdf>
- Sumcheck/MLE/HyperPlonk SageMath tutorial: <https://blog.zksecurity.xyz/posts/sumcheck-tutorial/>

38.9 S-two 进阶

- S-two 仓库: <https://github.com/starkware-libs/stwo>
- S-two book benchmarks: <https://docs.starknet.io/learn/S-two-book/benchmarks>
- StarkWare proving record(S-two): <https://starkware.co/blog/starkware-new-proving-record/>

38.10 Trusted Setup

- ETH KZG ceremony specs: <https://github.com/ethereum/kzg-ceremony-specs>
- Perpetual Powers of Tau: <https://github.com/privacy-scaling-explorations/perpetualpowersoftau>
- a16z On-Chain Trusted Setup Ceremony: <https://a16zcrypto.com/posts/article/on-chain-trusted-setup-ceremony/>
- Powers-of-Tau to the People(IACR 2022/1592): <https://eprint.iacr.org/2022/1592.pdf>

后续模块 09-替代生态: Solana、Aptos、Sui、Cosmos、Polkadot 等替代生态同样在大量引入 ZK 技术: Solana 的 Light Protocol(zk 隐私转账)、Aptos 的 ZK keyless 账户、Cosmos 的 IBC 跨链 zk light client。学完本模块的 ZK 基础,继续读第 09 模块可以看到这些技术在不同链生态里的具体落地形式。